Mohsin Iban Hossain

AIUB, AI MID Term Notes

ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEM

Table of Contents

INTRODUCTION TO ARTIFICIAL INTELLIGENCE (AI)

What is Artificial Intelligence?

⇨ Artificial Intelligence (AI) is the field of computer science that focuses on creating systems capable of performing tasks that typically require human intelligence. These include learning, reasoning, problem-solving, understanding language, perception, and decision-making.

Approaches to AI

Approach	Description
Acting Humanly	Machines that behave like humans (Turing Test approach).
Thinking Humanly	Machines that try to replicate human thought processes (Cognitive modeling).
Thinking Rationally	Machines that think logically and make rational decisions.
Acting Rationally	Machines that act to achieve the best outcome (Rational agent approach).

Turing Test

Proposed by: Alan Turing in 1950
Purpose: To determine if a machine can exhibit intelligent behavior equivalent to, or indistinguishable from, that of a human.

Test Setup:

- An interrogator communicates with a human and a machine via a text interface.
- If the interrogator can't reliably tell which is which, the machine is said to have passed the Turing Test.

Limitations:

- Focuses only on behavior, not internal reasoning.
- May allow non-intelligent tricks to fool the interrogator.

Foundations of AI

Discipline	Contribution to AI
Philosophy	Logic, reasoning, ethics, mind-body problem.
Mathematics	Algorithms, probability, optimization, formal models.
Psychology	Cognitive processes, learning models, decision-making.
Computer Science	Programming, complexity, data structures, software engineering.
Linguistics	Natural language understanding and generation.
Biology	Neural networks, genetic algorithms, behavior modeling.

Applications of AI

- **Expert Systems:** Decision support in medicine, engineering.
- **Natural Language Processing:** Chatbots, voice assistants.
- **Robotics:** Autonomous vehicles, drones.
- **Computer Vision:** Face recognition, object detection.
- **Machine Learning:** Spam filtering, fraud detection.
- **Games:** AI opponents in chess, Go, etc.

INTELLIGENT AGENTS & PROBLEM SOLVING

What is an Agent?

⇨ An **agent** is anything that perceives its environment through **sensors** and acts upon that environment through **actuators**.

Agent = **Perception → Action**

Agent and Environment

- **Environment:** The world or context in which the agent operates.
- **Sensors:** Devices that collect data from the environment (e.g., camera, temperature sensor).
- **Actuators:** Components that perform actions in the environment (e.g., motors, speakers).

Rational Agent

⇨ A **rational agent** is one that **does the right thing** — it chooses the action that maximizes performance, based on:

- Its **percept sequence**
- **Knowledge** of the environment
- **Possible actions**
- **Performance measure**

PEAS Description

⇨ To define a task environment, use the **PEAS** framework:

Component	Description
Performance	Criteria for success
Environment	What the agent senses
Actuators	What the agent uses to act
Sensors	What the agent uses to perceive the world

Example: Self-driving Car

Component	Description
Performance	Safety, speed, law compliance
Environment	Roads, other cars, pedestrians
Actuators	Steering, throttle, brake
Sensors	Camera, GPS, radar

Problem1: Describe the PEAS framework with an example of a Candy Jar Management Agent.

⇨ **PEAS** stands for **Performance measure, Environment, Actuators, and Sensors**. It is a framework used to define the task environment of an intelligent agent.

Let's apply the PEAS framework to a **Candy Jar Management Agent** – a system that automatically monitors and refills candies in a jar.

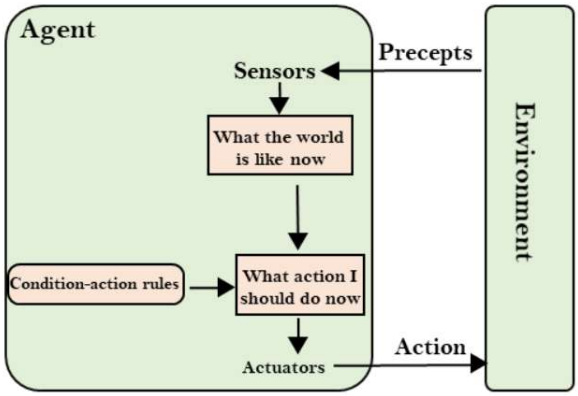
Example: Candy Jar Management Agent

Component	Description
Performance	Accuracy, Speed, Safety, Efficiency
Environment	Conveyor, Sorting Table, Jars, Lighting
Actuators	Robotic Arm, Conveyor Motor, Jar Mechanism
Sensors	Camera, Weight Sensor, Position Sensor, Proximity Sensor

Types of Agents

Type	Description
Simple Reflex Agent	Acts only on current percept; ignores history.
Model-Based Reflex Agent	Maintains internal state for decision-making.
Goal-Based Agent	Considers future consequences to achieve goals.
Utility-Based Agent	Uses a utility function to maximize overall happiness or benefit.
Learning Agent	Improves performance over time based on past experiences.

Simple Reflex Agent

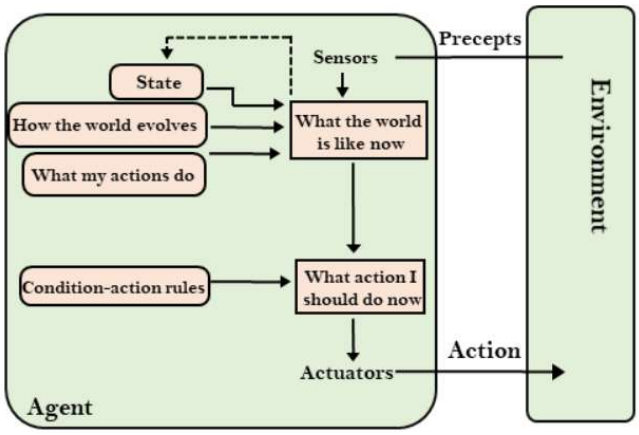


⇨ A simple reflex agent comprises the following parts:

- **Agent:** The agent is the one who performs actions on the environment.
- **Environment:** The environment includes the surroundings of the agent.
- **Actuators:** Actuators are devices that convert energy into motion.
- **Sensors:** Sensors are the things that sense the environment. They are devices that measure physical property.

```
function REFLEX-VACUUM-AGENT( [location,status]) returns an action
    if status = Dirty then return Suck
    else if location = A then return Right
    else if location = B then return Left
```

Model Based Agent



⇨ The model-based reflex agent operates in four stages:

- **Sense:** It perceives the current state of the world with its sensors.
- **Model:** It constructs an internal model of the world from what it sees.
- **Act:** The agent carries out the action that it has chosen.
- **Reason:** It uses its model of the world to decide how to act based on a set of predefined rules or heuristics.

Example: A vacuum cleaner that uses sensors to detect dirt and obstacles and moves and cleans based on a model.

```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
              model, a description of how the next state depends on current state and action
              rules, a set of condition–action rules
              action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Figure 2.12 A model-based reflex agent. It keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent.

Goal Based Agent

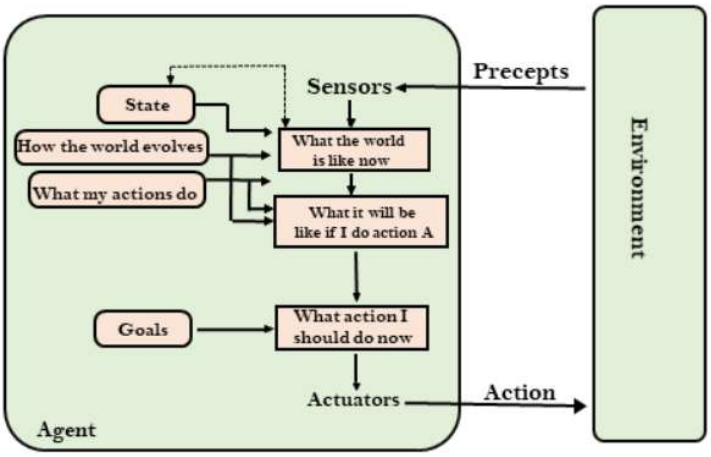


Figure 2.13 A model-based, goal-based agent. It keeps track of the world state as well as a set of goals it is trying to achieve, and chooses an action that will (eventually) lead to the achievement of its goals.

Utility Based Agent

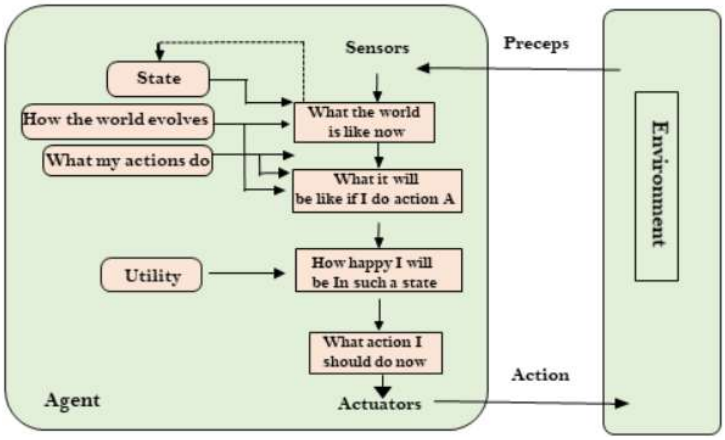
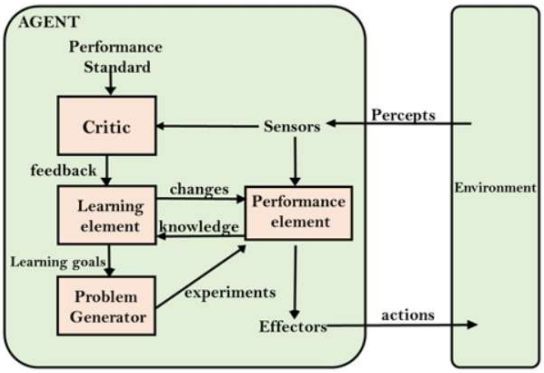


Figure 2.14 A model-based, utility-based agent. It uses a model of the world, along with a utility function that measures its preferences among states of the world. Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.

Learning Based Agent



HOW COMPONENT OF AGENT PROGRAMS WORK

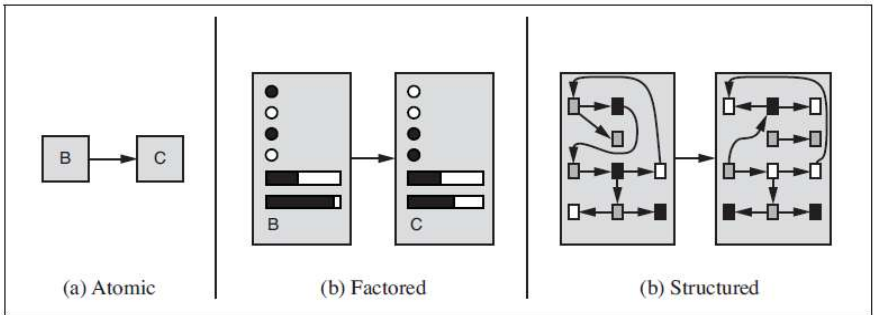


Figure 2.16 Three ways to represent states and the transitions between them. (a) Atomic representation: a state (such as B or C) is a black box with no internal structure; (b) Factored representation: a state consists of a vector of attribute values; values can be Boolean, real-valued, or one of a fixed set of symbols. (c) Structured representation: a state includes objects, each of which may have attributes of its own as well as relationships to other objects.

Problem-Solving Agent

⇨ A **problem-solving agent** decides what to do by searching through **possible sequences of actions** that lead to the goal.

Steps in Problem Formulation:

1. **Initial state**
2. **Actions**
3. **Transition model**
4. **Goal test**
5. **Path cost**

Together these define a **search problem**.

Example: Vacuum Cleaner World

- **States:** Location and status (clean/dirty)
- **Actions:** Move Left, Move Right, Suck
- **Goal:** Clean all locations
- **Performance:** Number of clean squares, energy used

UNINFORMED & INFORMED SEARCH TECHNIQUES

Problem Solving by Searching

⇨ A **search algorithm** explores a **state space** to find a **path from the initial state to the goal state**.

State Space = All possible configurations
Search Tree = Path from initial to goal through actions
Search Node = Contains a state, path cost, and parent

Uninformed Search (Blind Search)

⇨ Uninformed search strategies **do not use any domain-specific knowledge**. They rely purely on the structure of the problem.

Algorithm	Description
Breadth-First Search (BFS)	Explores nodes level by level. Guaranteed to find the shortest path in an unweighted graph.
Depth-First Search (DFS)	Explores as deep as possible before backtracking. Can go into infinite loops without limit.
Uniform Cost Search (UCS)	Expands the node with the lowest path cost (like Dijkstra's). Optimal and complete.
Depth-Limited Search (DLS)	DFS with a depth limit. Avoids infinite loops.
Iterative Deepening Search (IDS)	Repeated DLS with increasing depth. Combines space efficiency of DFS and completeness of BFS.
Bidirectional Search	Simultaneously searches forward from the start and backward from the goal. Very fast when applicable.

Informed Search (Heuristic Search)

⇨ Informed search algorithms use **heuristic functions (h(n))** to guide the search. These functions estimate the cost from node **n** to the goal.

◇ Best-First Search

- Chooses the node with the **lowest heuristic value (h(n))**
- Not guaranteed to be optimal

◇ A* Search

- Uses **f(n) = g(n) + h(n)**
  where:
  - g(n): Cost to reach node n
  - h(n): Estimated cost from n to goal
- **Complete and Optimal**, if h(n) is **admissible** (never overestimates) and **consistent** (monotonic).

Example Heuristics

- **Manhattan Distance:** For grid-based problems.
- **Euclidean Distance:** For map or spatial problems.
- **Number of misplaced tiles:** For puzzle-solving.

Comparison of Search Strategies

Algorithm	Time	Space	Optimal?	Complete?
BFS	Exponential	Exponential	Yes	Yes
DFS	$O(b^m)$	O(m)	No	No
UCS	$O(b^d)$	$O(b^d)$	Yes	Yes
DLS	$O(b^l)$	O(l)	No	Yes (if $l \geq d$)
IDS	$O(b^d)$	O(d)	Yes	Yes
A*	Depends on h(n)	Depends	Yes (if h is admissible)	Yes

b = branching factor
d = depth of the shallowest solution
m = maximum depth
l = limit

BREADTH-FIRST SEARCH (BFS)

Breadth-First Search (BFS)

⇨ **Breadth-First Search (BFS)** is an uninformed search algorithm that explores the **shallowest** (closest to the root) **nodes first**, level by level.

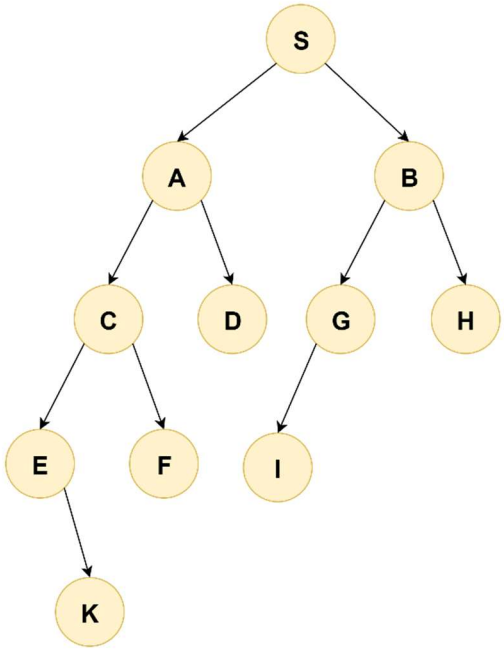
Note: It uses a **queue (FIFO)** data structure to keep track of the next nodes to visit.

Algorithm Steps

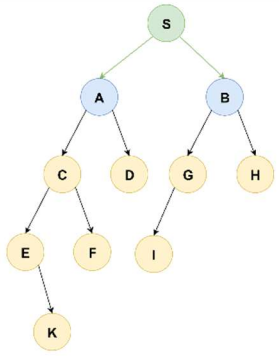
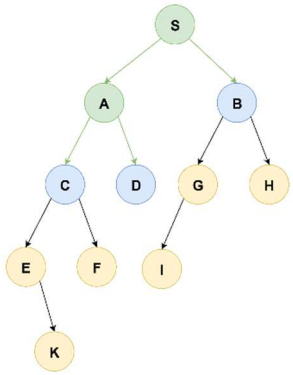
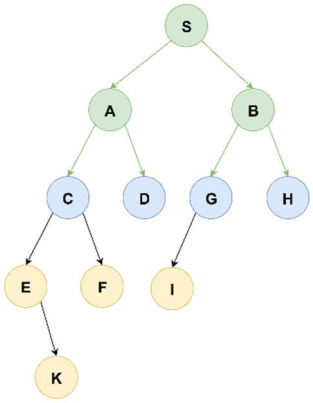
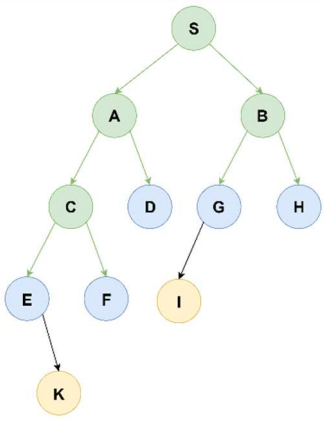
1. **Initialize** the queue with the **initial state**.
2. **Repeat** until the queue is empty:
   - Remove the **front node**.
   - If it is the **goal**, return the solution.
   - Otherwise, expand the node (generate its children).
   - **Add the children to the end of the queue**.

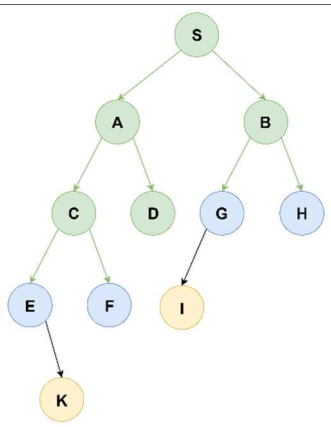
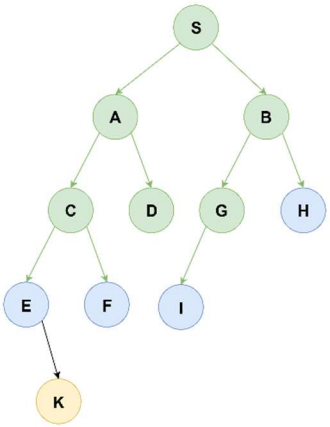
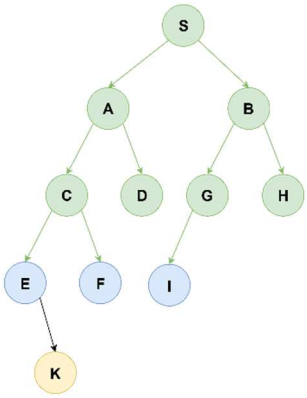
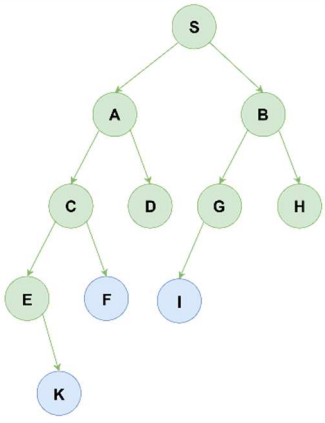
Problem1:

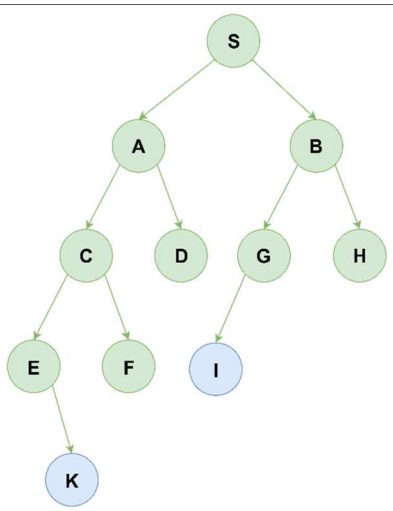
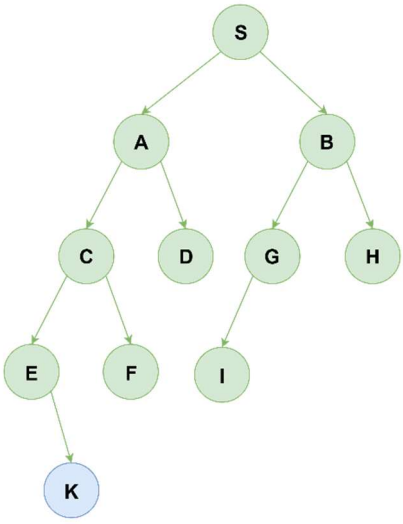
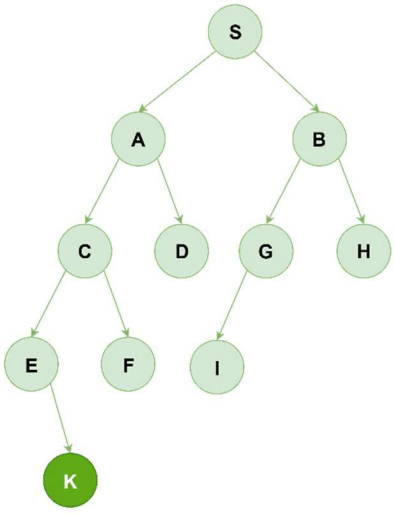
≫From the following graph to solve the problem where **S is the start node** and **K is the goal node**. Note that the **edges in this graph are directed**. For this problem Find the Goal node using Breadth-First Search (BFS).



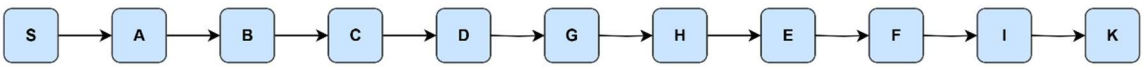
Solutions:

Vertex Visited	Vertex in Queue	Graph
S	A B	
S A	B C D	
S A B	C D G H	
S A B C	D G H E F	

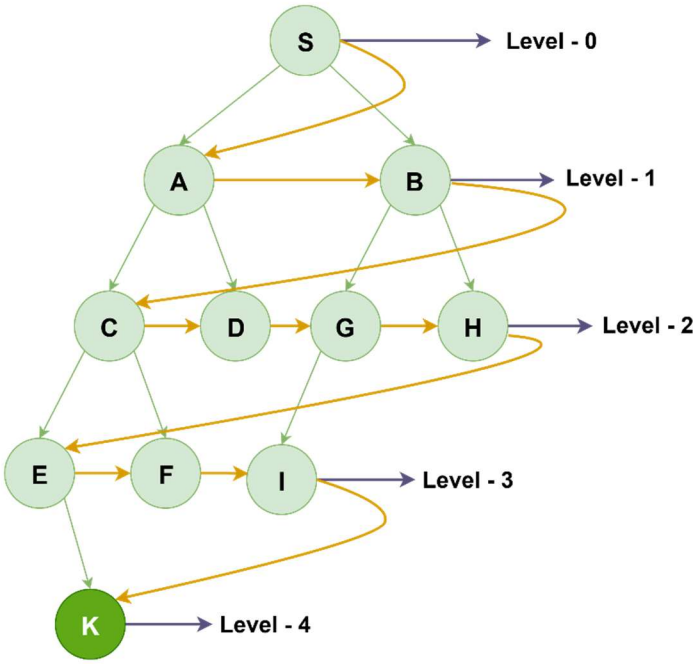
S A B C D	G H E F	
S A B C D G	H E F I	
S A B C D G H	E F I	
S A B C D G H E	F I K	

S A B C D G H E F	I K	
S A B C D G H E F I	K	
S A B C D G H E F I K		

⇨ **Now the path is:**



How its work

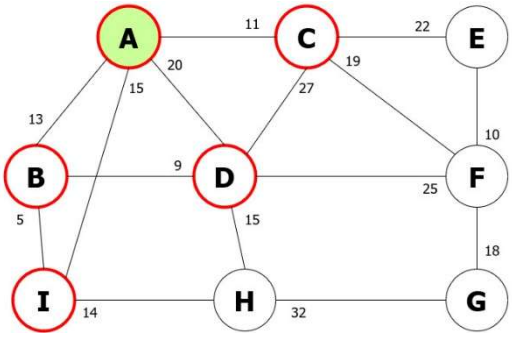
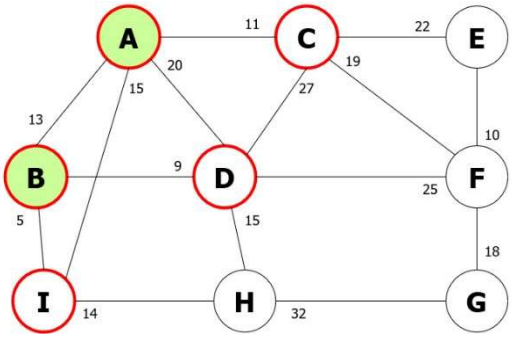
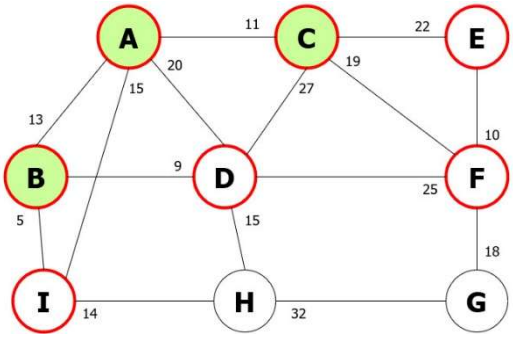


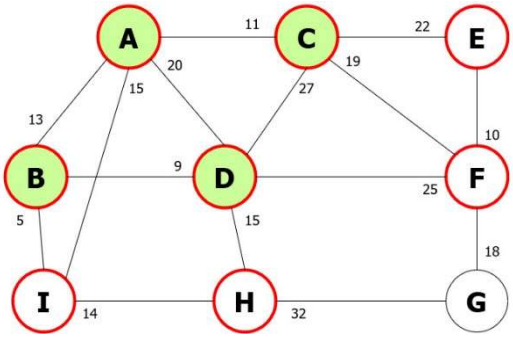
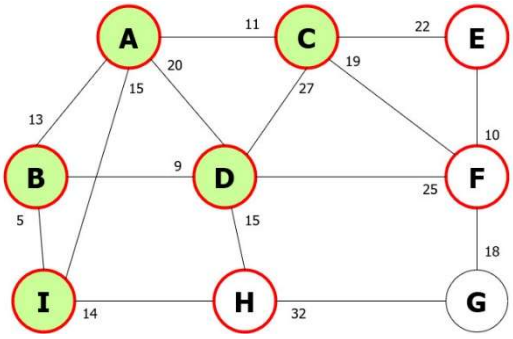
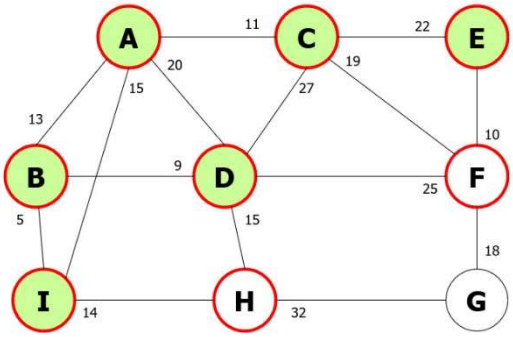
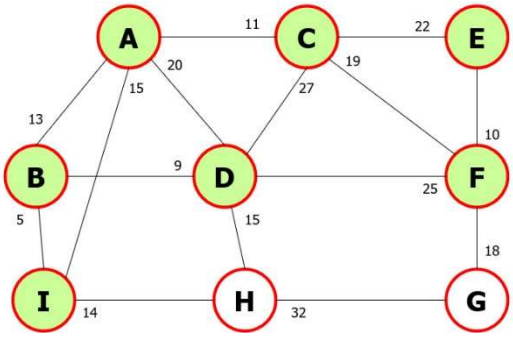
Properties

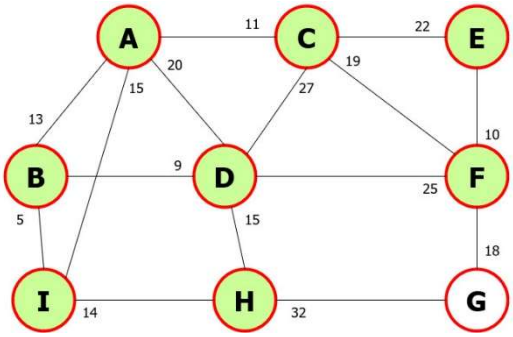
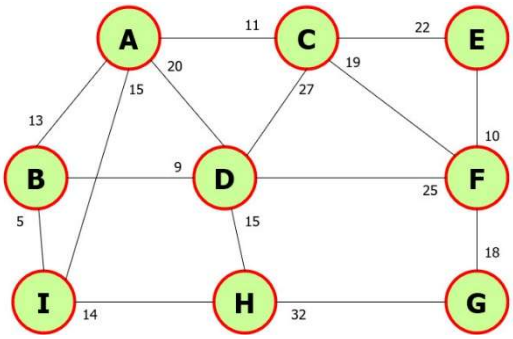
Property	Description
Completeness	Yes, if branching factor is finite.
Optimality	Yes, if all step costs are equal.
Time Complexity	$O(b^d)$, where b = branching factor, d = depth of solution.
Space Complexity	$O(b^d)$ – because it stores all nodes in memory.

Problem2:

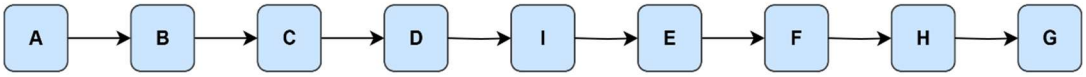
≫ Simulate Breadth-First Search (BFS) on the graph shown in below, **starting from the node A** and **aiming to reach the goal node G**. (push nodes into the frontier in alphabetic order)

Vertex Visited	Vertex in Queue	Graph
A	B C D I	
A B	C D I	
A B C	D I E F	

A B C D	I E F H	
A B C D I	E F H	
A B C D I E	F H	
A B C D I E F	H G	

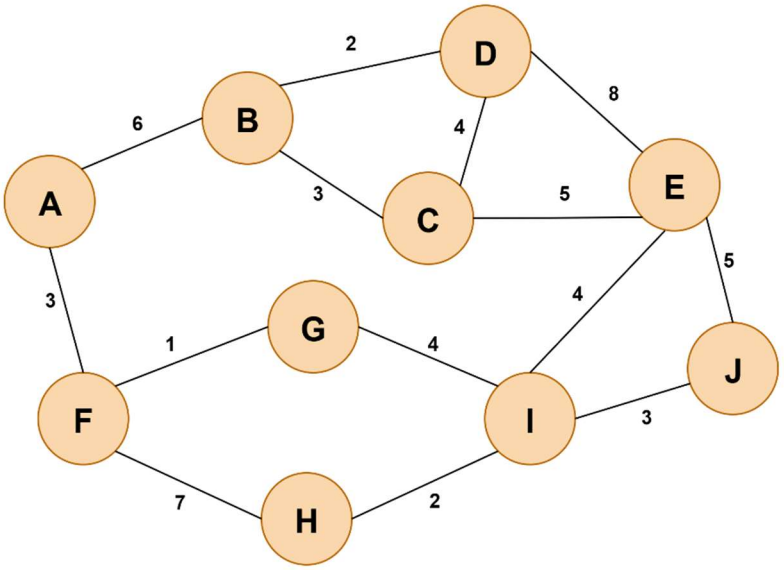
A B C D I E F H	G	
A B C D I E F H G		

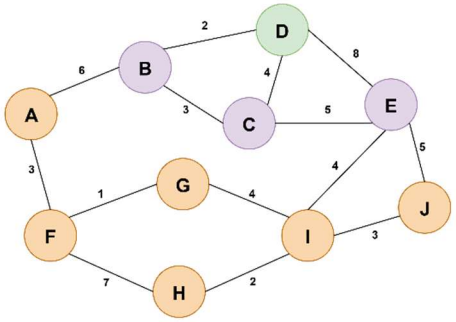
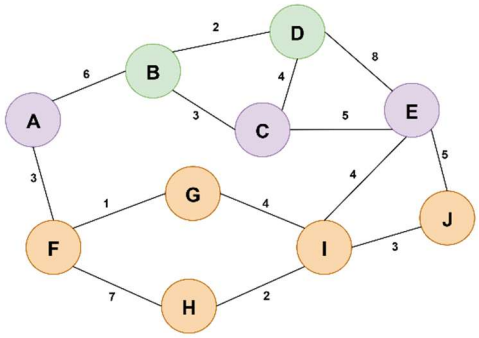
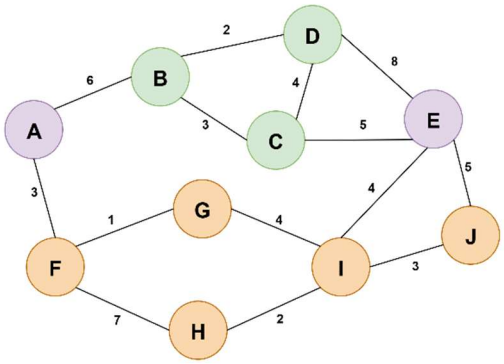
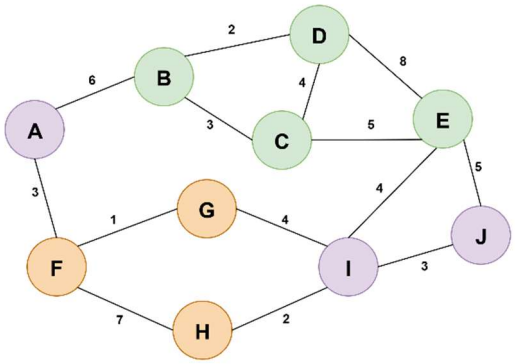
⇨ **Now the path is:**

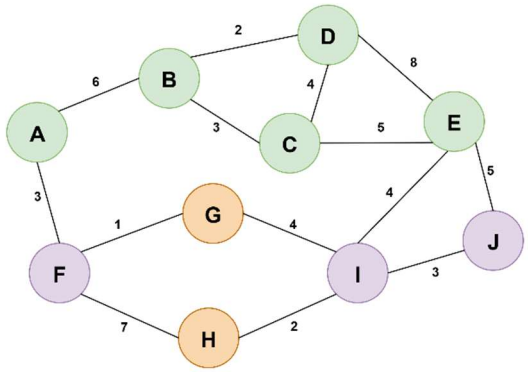
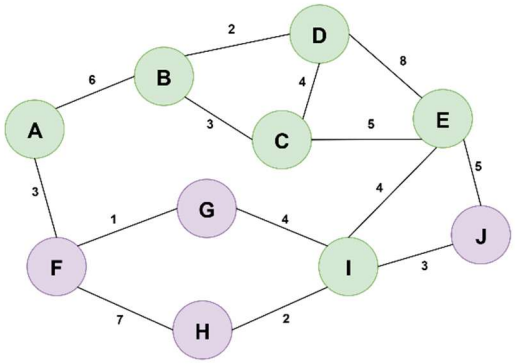
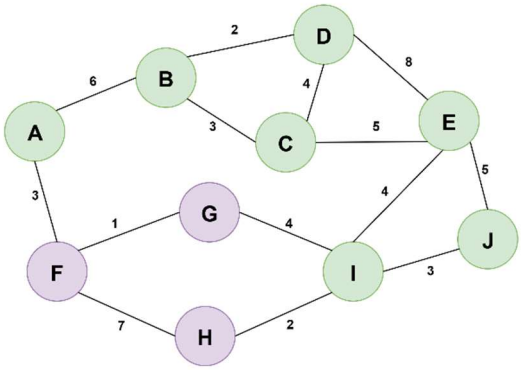
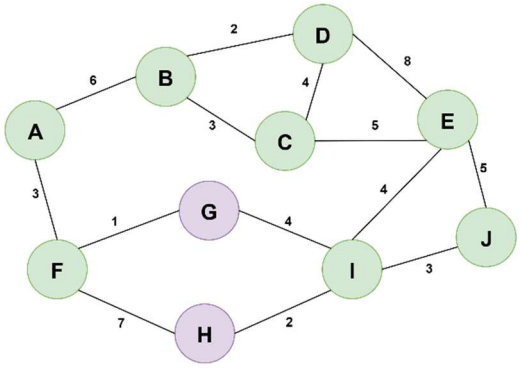


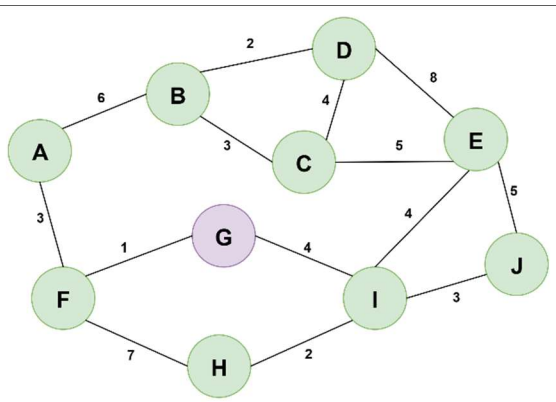
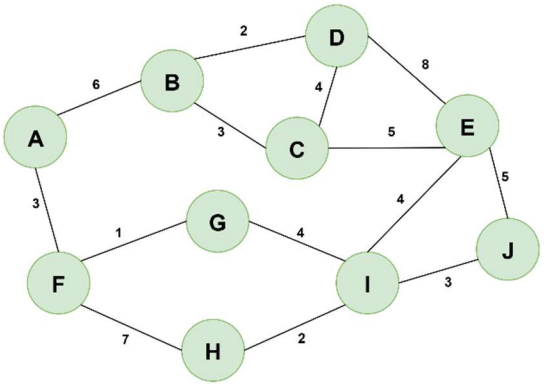
☌ Problem3:

≫From the following graph to solve the problem where **destination node G Stating from node D**. Note that the **edges in this graph are undirected**. For this problem Find the Goal node using Breadth-First Search (BFS).

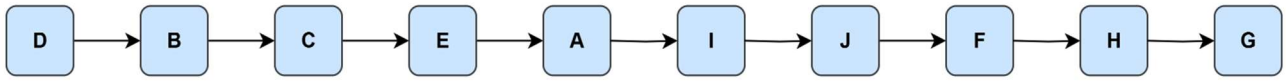


Vertex Visited	Vertex in Queue	Graph
D	B C E	
D B	C E A	
D B C	E A	
D B C E	A I J	

D B C E A	I J F	
D B C E A I	J F H G	
D B C E A I J	F H G	
D B C E A I J F	H G	

D B C E A I J F H	G	
D B C E A I J F H G		

⇨ **Now the path is:**



Properties

Property	Description
Completeness	Yes, if branching factor is finite.
Optimality	Yes, if all step costs are equal.
Time Complexity	$O(b^d)$, where b = branching factor, d = depth of solution.
Space Complexity	$O(b^d)$ – because it stores all nodes in memory.

DEPTH-FIRST SEARCH (DFS)

Depth-First Search (DFS)

⇨ **Depth-First Search (DFS)** explores **as deep as possible** along a branch before **backtracking**.

Note: It uses a **stack (LIFO)** data structure or recursion.

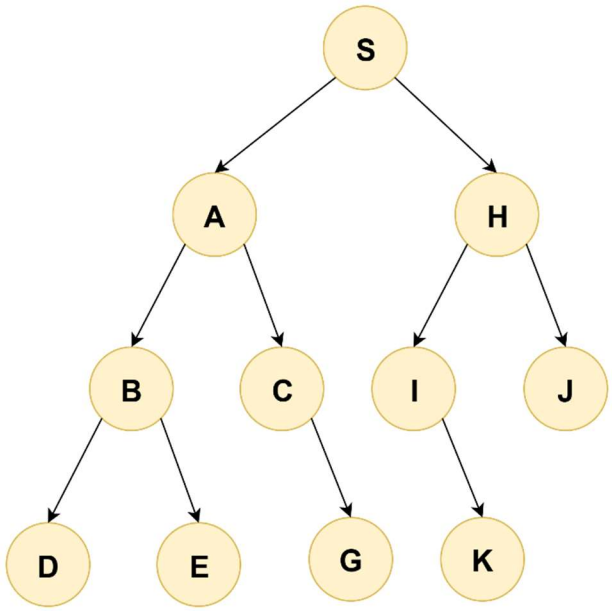
Algorithm Steps

1. Start from the **initial node**.
2. Push the node onto a **stack**.
3. While the stack is not empty:

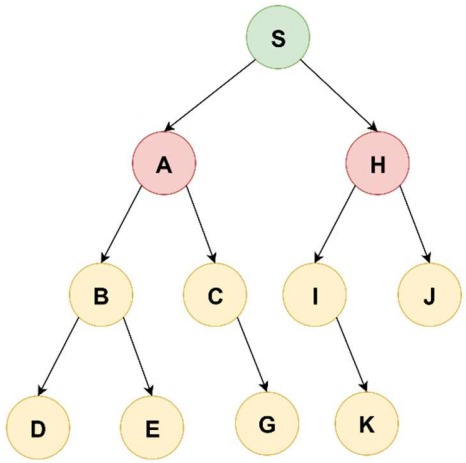
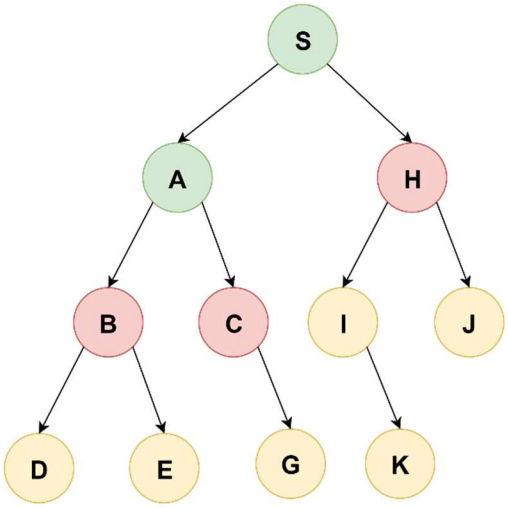
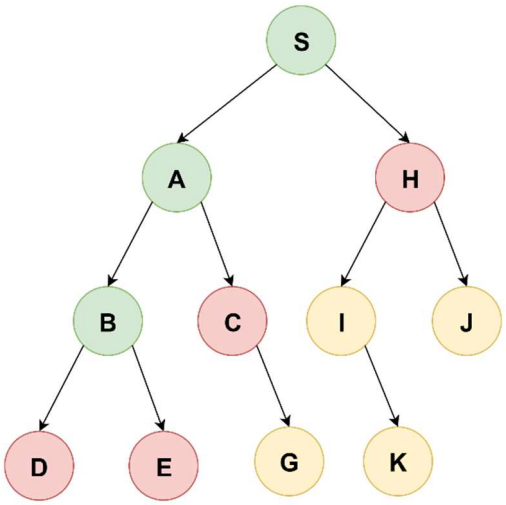
    - Pop the top node.
    - If it's the goal, return success.
    - Else, expand it and **push all its unvisited children** onto the stack.

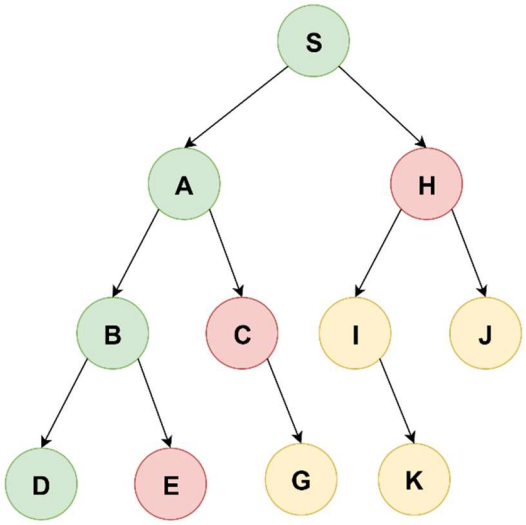
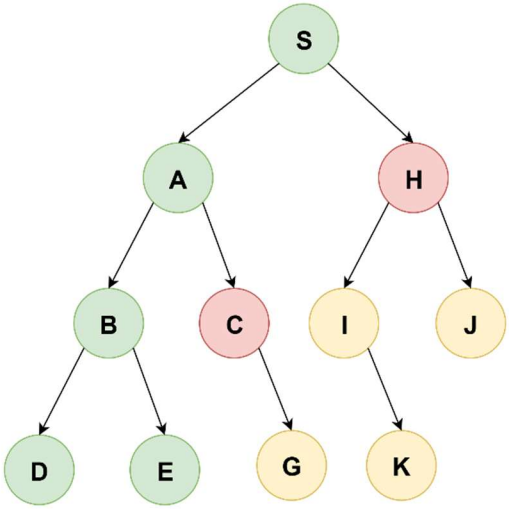
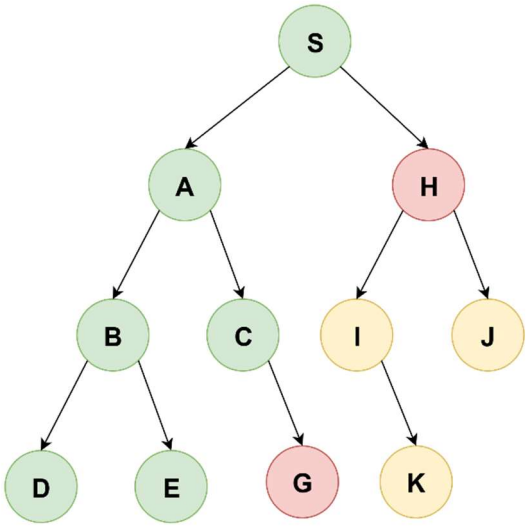
Problem1:

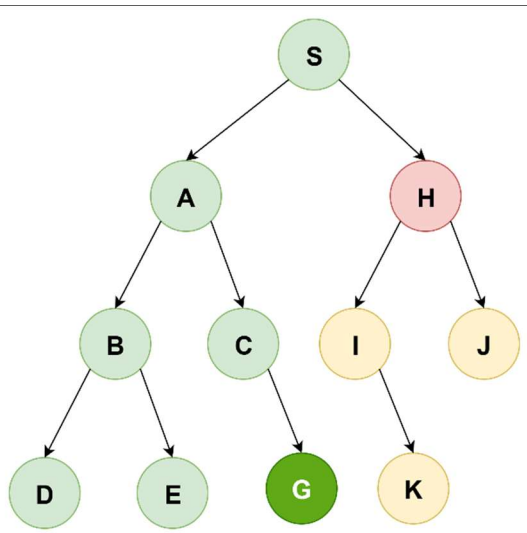
≫From the following graph to solve the problem where **S is the start node** and **G is the goal node**. Note that the **edges in this graph are directed**. For this problem Find the Goal node using Depth-First Search (DFS).



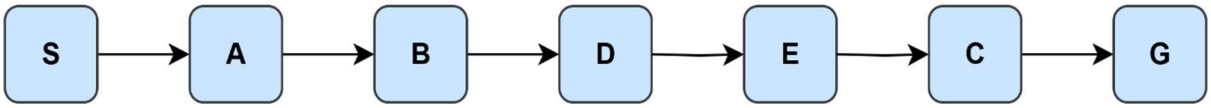
⇨ Solutions:

Vertex Visited	Vertex in Stack	Graph
S	A H	
S A	B C H	
S A B	D E C H	

S A B D	E C H	
S A B D E	C H	
S A B D E C	G H	

S A B D E C G	H	

⇨ **Now the path is:**

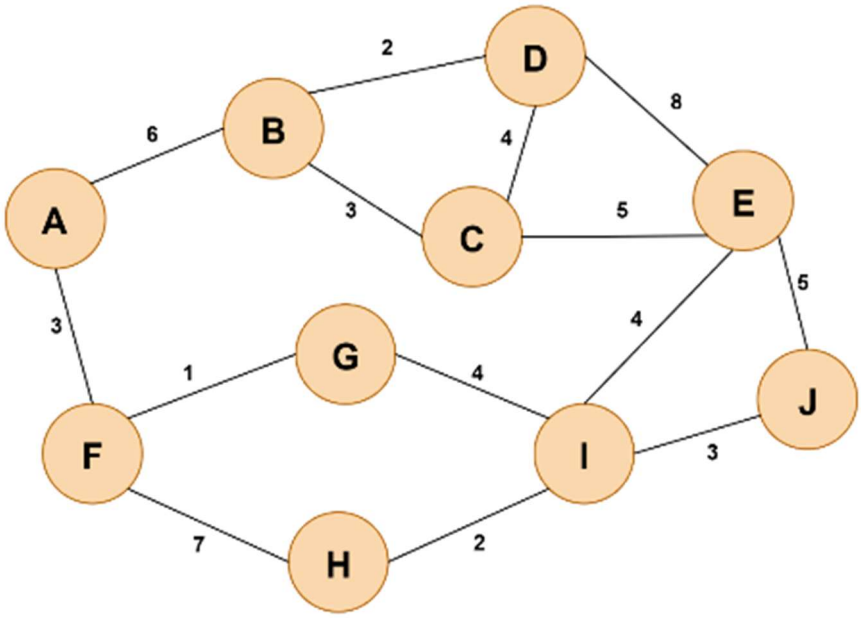


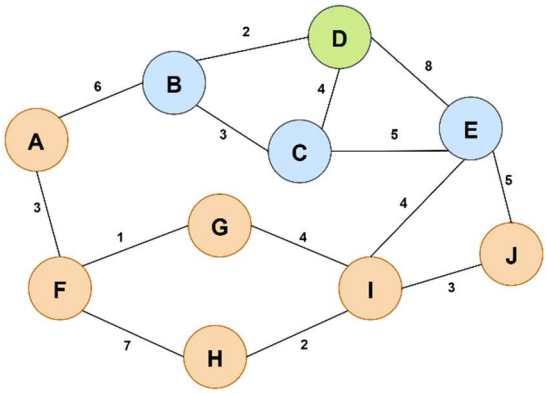
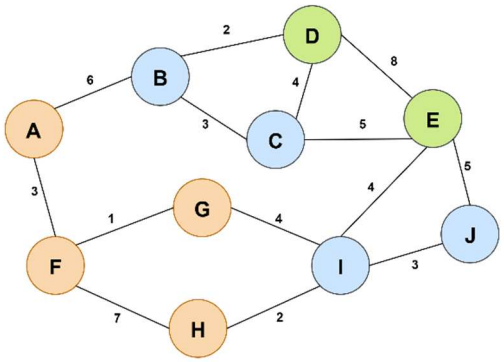
Properties of DFS

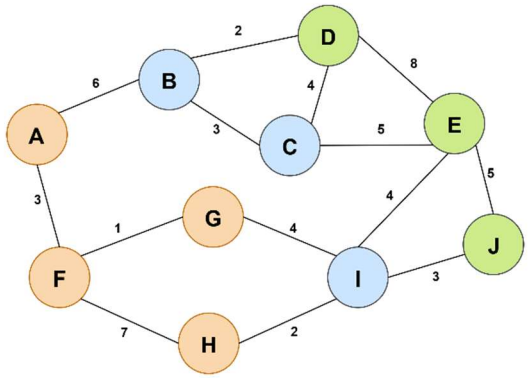
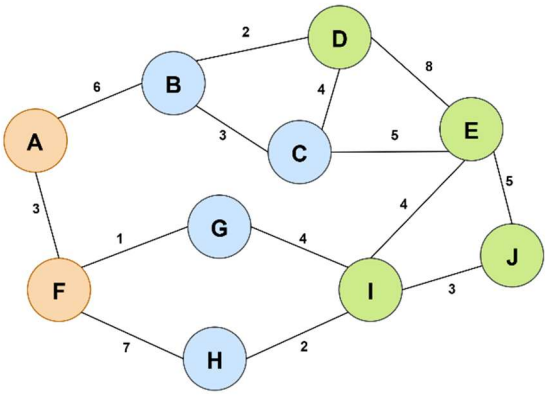
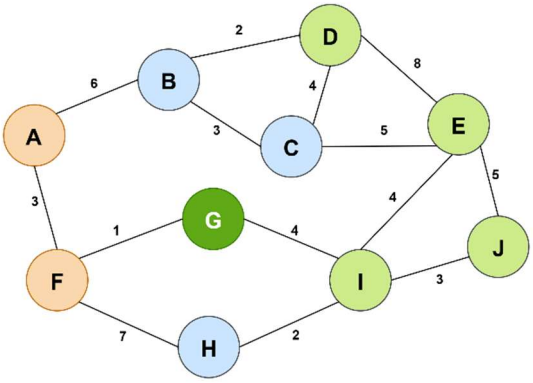
Property	Value
Complete	❌ No (fails in infinite-depth spaces)
Optimal	❌ No (may return suboptimal path)
Time	$O(b^m)$, where b = branching factor, m = max depth
Space	O(m) for recursive DFS; $O(b^m)$ for iterative
Best for	Problems with **deep solutions** or **low memory**

Problem2:

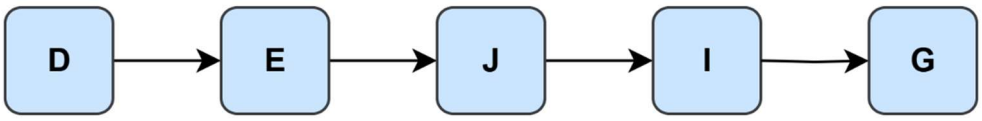
≫From the following graph to solve the problem where **D is the start node** and **G is the goal node**. Note that the **edges in this graph are undirected**. For this problem Find the Goal node using Depth-First Search (DFS).



Vertex Visited	Vertex in Stack	Graph
D	E C B	
D E	J I C B	

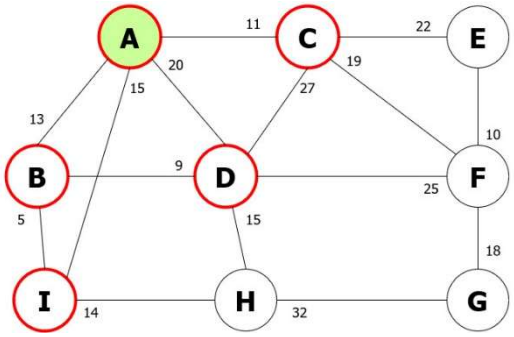
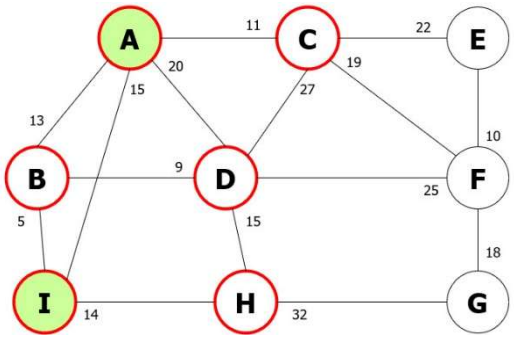
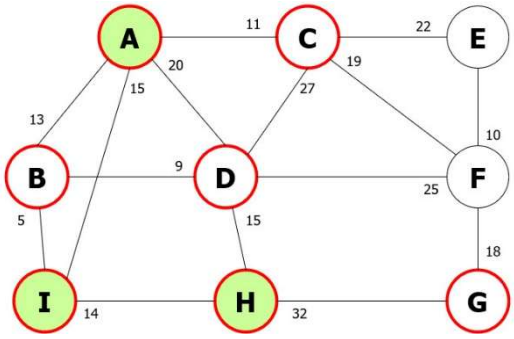
D E J	I C B	
D E J I	G H C B	
D E J I G	F H C B	

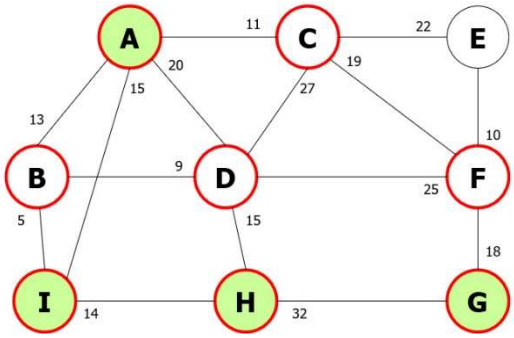
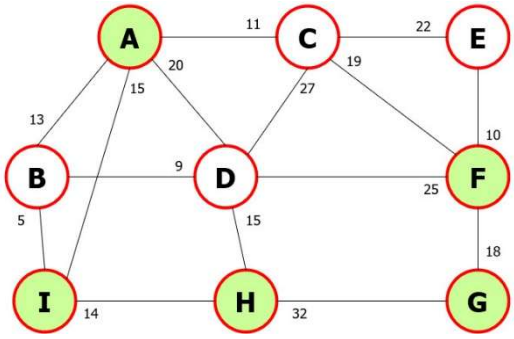
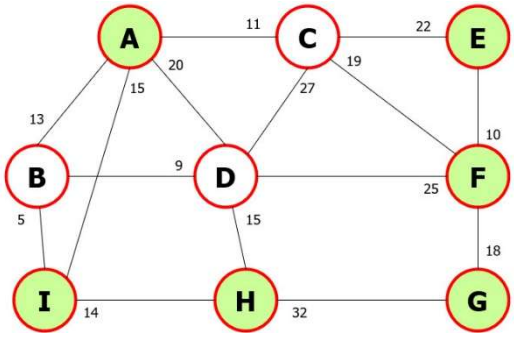
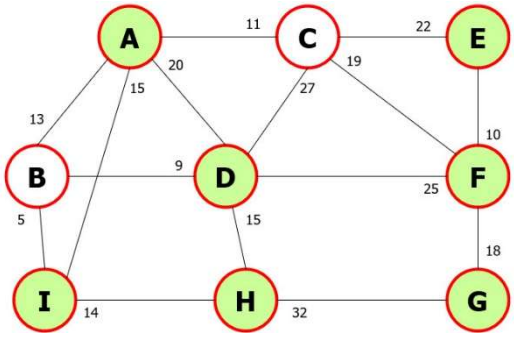
⇨ **Now the path is:**

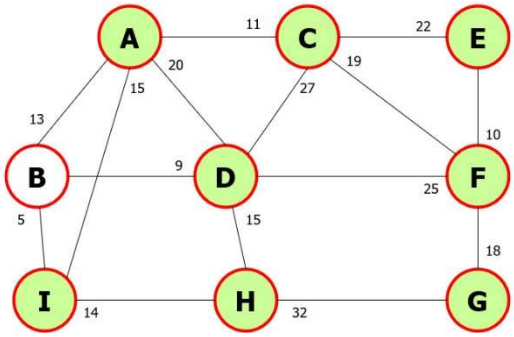
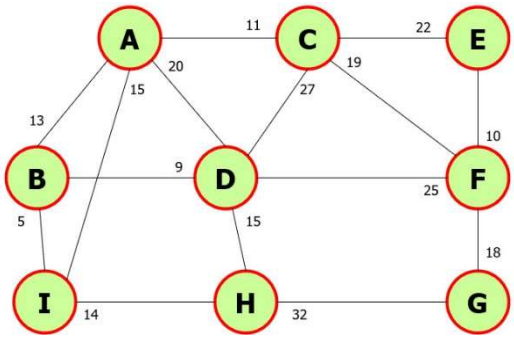


Problem3:

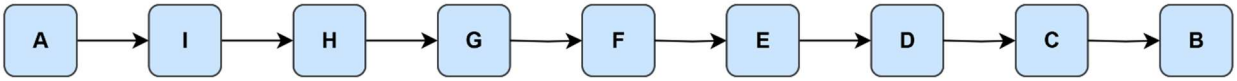
≫Simulate Depth-First Search (DFS) on the graph shown in below, **starting from the node A** and **aiming to reach the goal node B**. (push nodes into the frontier in alphabetic order)

Vertex Visited	Vertex in Stack	Graph
A	I D C B	
A I	H D C B	
A I H	G D C B	

A I H G	F D C B	
A I H G F	E D C B	
A I H G F E	D C B	
A I H G F E D	C B	

A I H G F E D C	B	
A I H G F E D C B		

⇨ **Now the path is:**



UNIFORM COST SEARCH (UCS)

Uniform Cost Search (UCS)

⇨ **Uniform Cost Search (UCS)** is an **uninformed** search algorithm that expands the node with the **lowest total path cost** so far.

Note: UCS is a generalization of BFS that works on **weighted graphs** (with **different edge costs**).

⇨ **Key Concepts:**

- It uses a **priority queue (min-heap)** to always expand the **cheapest path**.
- Nodes are prioritized by `g(n)`, the **cost from the start node to node n**.
- UCS ignores the number of steps or depth and focuses only on cost.

Algorithm Steps

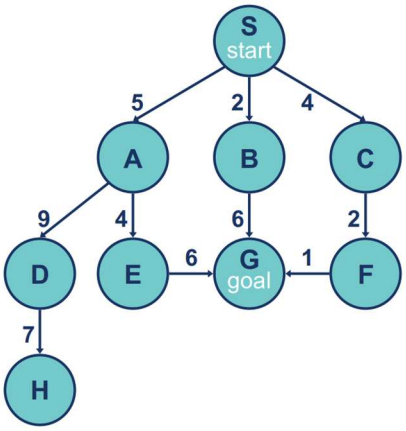
1. Initialize a **priority queue** with the initial node (cost = 0).
2. Loop until the queue is empty:
   - Remove the node with the **lowest path cost**.
   - If it's the **goal**, return the path.
   - Else, **expand** the node:
     - For each child, calculate path cost `g(child)`
     - If child is unvisited or has a lower cost now, **update the queue**.

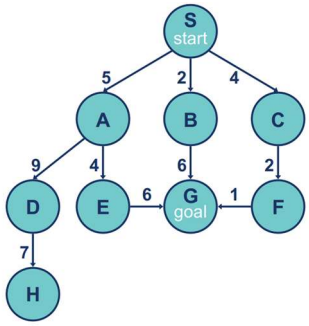
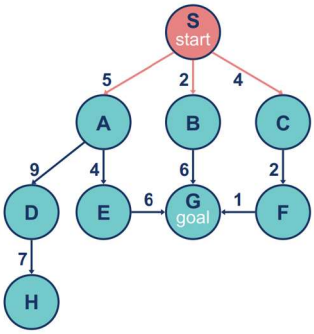
UCS vs BFS

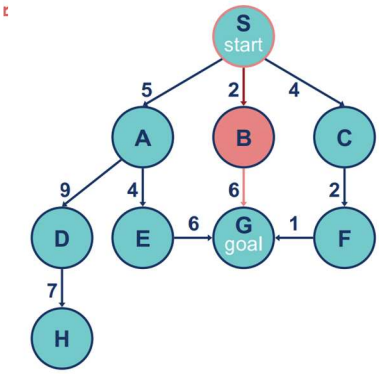
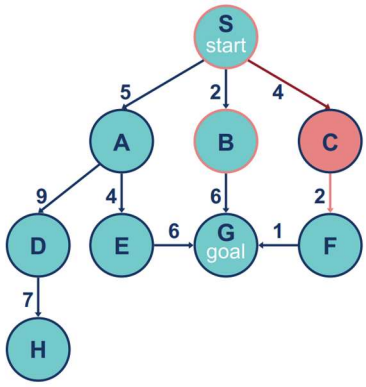
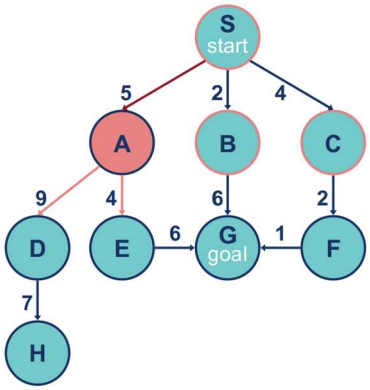
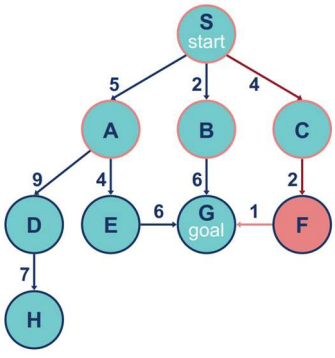
Feature	UCS	BFS
Edge Cost	Can handle **different costs**	Assumes **equal cost**
Priority	Based on **path cost** (`g(n)`)	Based on **depth/level**
Data Structure	**Priority Queue**	**FIFO Queue**
Optimality	**Optimal** (guaranteed)	If all costs are equal
Completeness	**Complete**	Complete

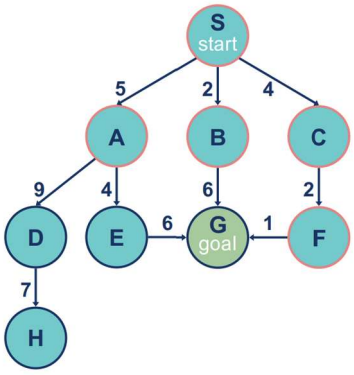
Problem1:

≫From the following graph to solve the problem where you need to reach the **destination node G & starting from node S.** Apply **Uniform Cost Search (UCS).** Show the steps of searching for the best path.

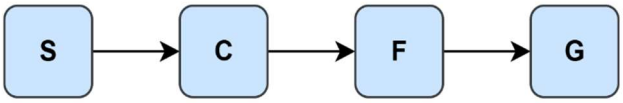


Frontier List	Expand List	Graph Explored
{(S,0)}	S	
{(S-B,2)} {(S-C,4)} {(S-A,5)}	B	

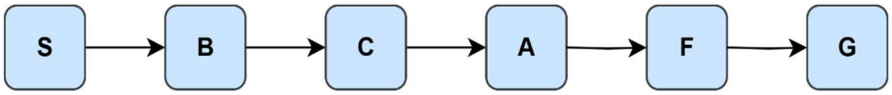
{(S-C,4)} {(S-A,5)} {(S-B-G,8)}	C	
{(S-A,5)} {(S-B-G,8)} {(S-C-F,6)}	A	
{(S-B-G,8)} {(S-C-F,6)} {(S-A-E,9)} {(S-A-D,14)}	F	
{(S-B-G,8)} {(S-A-E,9)} {(S-A-D,14)} {(S-C-F-G,7)}	G goal	

{(S-B-G,8)}	{(S-A-E,9)}	No Expend	
{(S-A-D,14)}	{(S-C-F-G,7)}		

⇨ **Actual path is:**



⇨ **Traversed path:**



⇨ **Actual Cost:**

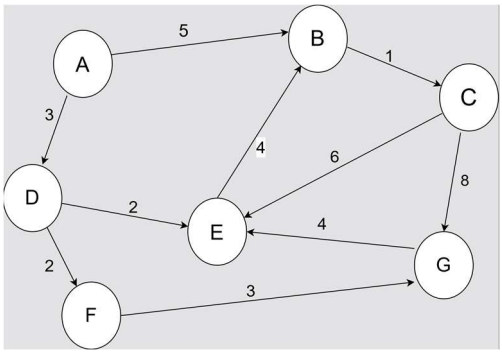
Cost: 7

Properties of UCS

Property	Value
Complete	Yes (if step costs $\geq \varepsilon > 0$)
Optimal	Yes (finds the least-cost path)
Time	**$O(b^{(1 + C^*/\varepsilon)})$,** where C^* is optimal cost, ε is smallest edge cost
Space	Can be large (stores all expanded paths)

Problem2:

≫From the following graph to solve the problem where you need to reach the **destination node G & starting from node A.** Apply **Uniform Cost Search (UCS).** Show the steps of searching for the best path.



Step	Frontier List	Expand List	Explored List
1	{(A,0)}	A	NULL
2	{(A-**D**, 3), (A-B, 5)}	D	{A}
3	{(A-**B**, 5), (A-D-E, 5), (A-D-F, 5)}	B	{A, D}
4	{(A-D-**E**, 5), (A-D-F, 5), (A-B-C, 6)}	E	{A, D, B}
5	{(A-D-**F**, 5), (A-B-C, 6), ~~(A-D-E-B, 9)~~} *here **B** is **already explored**	F	{A, D, B, E}
6	{(A-B-**C**, 6), (A-D-F-G,8)}	C	{A, D, B, E, F}
7	{(A-D-F-**G**,8),~~(A-B-C-E,12)~~, (A-B-C-G, 14)} *here **E** is **already explored**	G	{A, D, B, E, F, C}
8	{(A-D-F-G,8)}	NULL	{A, D, B, E, F, C, **G**} **# GOAL Found!**

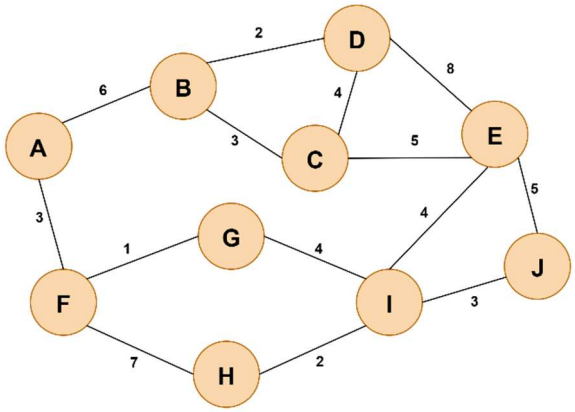
⇨ **Actual path is:** A → D → F → G

⇨ **Traversed path:** A → D → B → E → F → C → G

⇨ **Actual Cost:** **Cost: 8**

Problem3:

≫From the following graph to solve the problem where you need to reach the **destination node G & starting from node D.** Apply **Uniform Cost Search (UCS).** Show the steps of searching for the best path.



Step	Frontier List	Expand List	Explored List
1	{(D,0)}	D	NULL
2	{(D-**B**, 2), (D-C, 4), (D-E, 8)}	B	{D}
3	{**(D-C, 4),** (D-E, 8), (D-**B-C**, 5), (D-**B-A**, 8)}	C	{D, B}
4	{(D-E, 8), ~~(D-**B-C**, 5)~~, (D-**B-A**, 8), ~~(D-C-B,7)~~, (D-C-E,9)}	E	{D, B, C}
5	{(D-**B-A**, 8), ~~(D-C-E,9)~~ (D-E-I, 12), (D-E-J, 13),}	A	{D, B, C, E}
6	{(D-E-I, 12), (D-E-J, 13), (D-**B-A-F**, 11)}	F	{D, B, C, E, A}
7	{(D-E-I, 12), (D-E-J, 13), (D-**B-A-F-G**, 12), (D-**B-A-F-H**, 18)}	G	{D, B, C, E, A, F}
8	(D-**B-A-F-G**, 12)	NULL	{D, B, C, E, A, F, G} **# GOAL Found!**

⇨ **Actual path is:** D → B → A → F → G

⇨ **Traversed path:** D → B → C → E → A → F → G

⇨ **Actual Cost:** **Cost: 12**

DEPTH-LIMITED SEARCH (DLS)

Depth-Limited Search (DLS)

⇨ **Depth-Limited Search (DLS)** is a version of **Depth-First Search (DFS)** where the search is **limited to a predefined depth (ℓ)**.

Note: It helps **avoid infinite loops** in graphs or trees with cycles or infinite depth.

DFS can fail in:

- Infinite trees or graphs
- Cycles

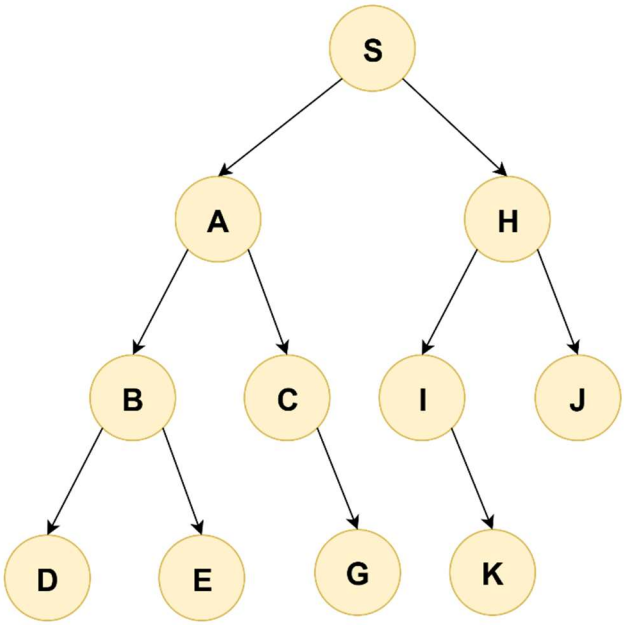
By limiting the depth of exploration, **DLS provides more control** and helps avoid such failures.

Algorithm Steps

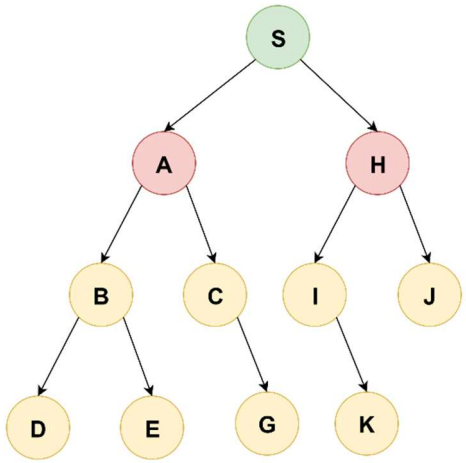
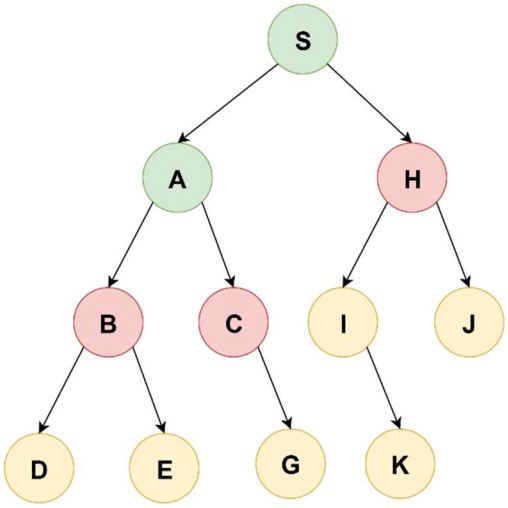
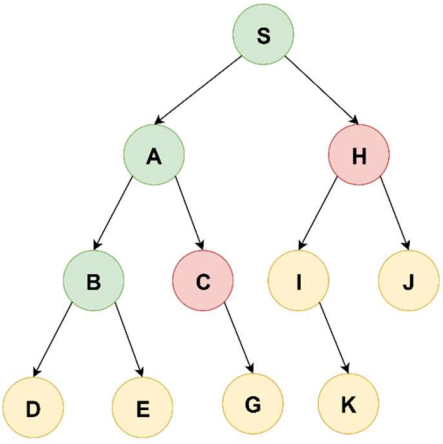
1. Start from the initial node.
2. Traverse the search tree **recursively**, tracking the current depth.
3. If the current depth exceeds the limit ℓ, **cut off** that path.
4. Continue until the goal is found or all paths within the limit are explored.

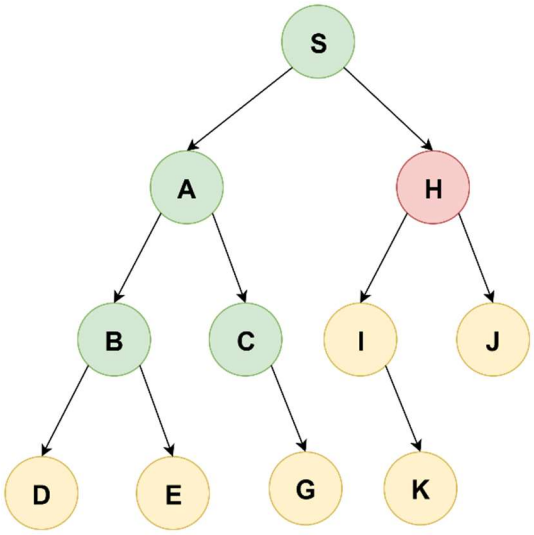
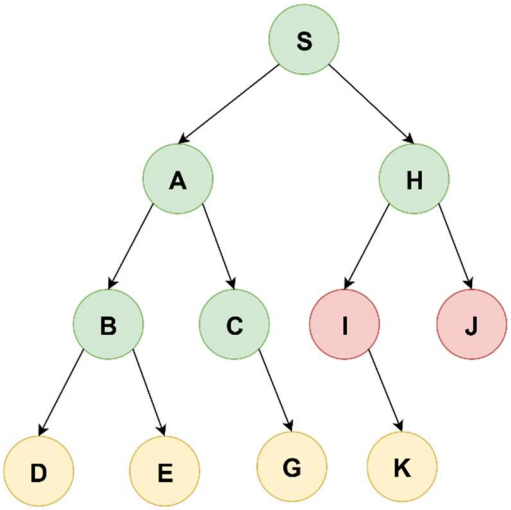
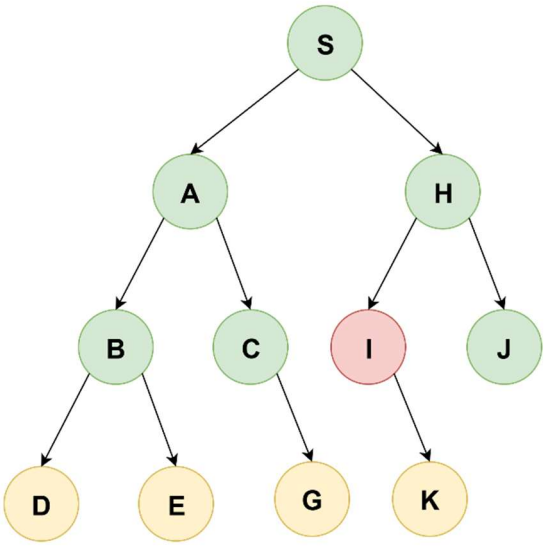
Problem1:

≫From the following graph to solve the problem where **S is the start node,** and **I is the goal node**. Note that the **edges in this graph are directed**. For this problem Goal node using Depth-**Limited** Search (DLS) Where and the depth limit is 2.

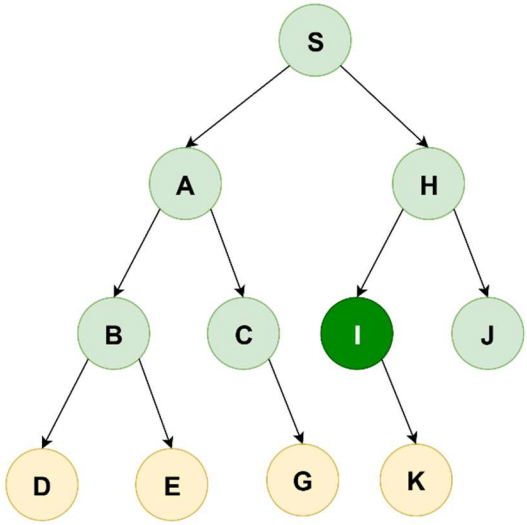


⇨ Solutions:

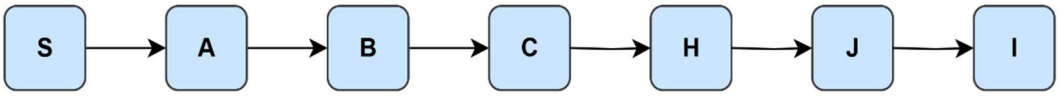
Vertex Visited	Vertex in Stack	Graph
S	A H	
S A	B C H	
S A B	C H	

S A B C	H	
S A B C H	J I	
S A B C H J	I	

S	A	B	C	H	J	I



⇨ **Now the path is:**

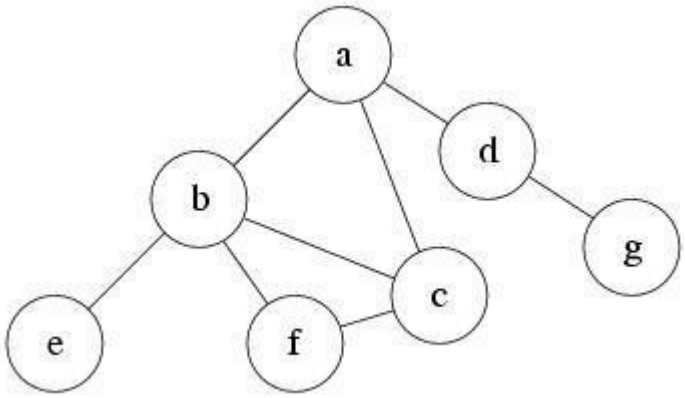


Properties of DLS

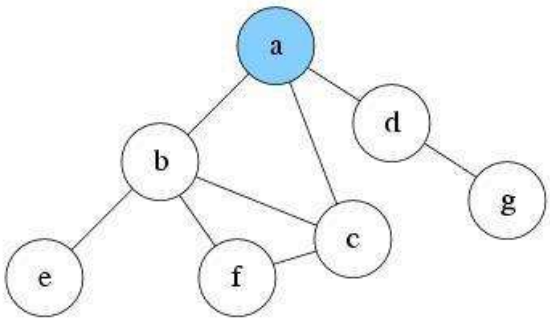
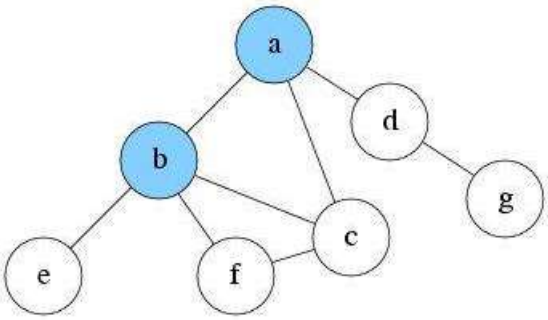
Property	Value
Completeness	No, if the solution is deeper than the limit
Optimality	No
Time	O(b^ℓ), where b = branching factor, ℓ = depth limit
Space	O(ℓ) (same as DFS – linear in depth)

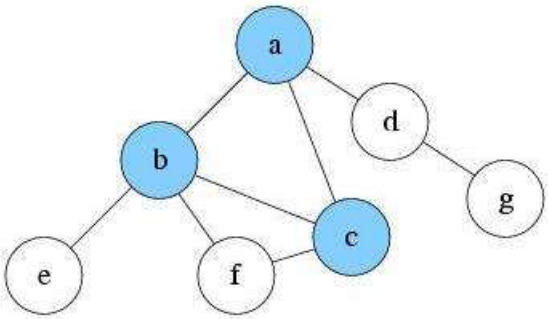
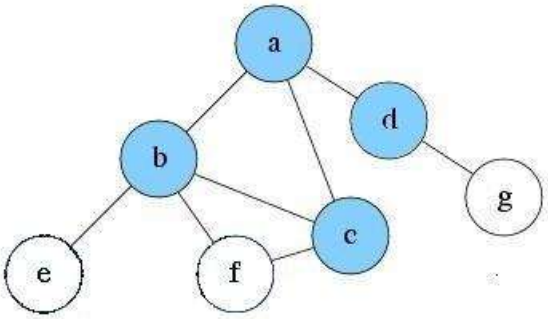
Problem2:

≫From the following graph to solve the problem where **'a' is the start node** and **'g' is the goal node**. Note that the **edges in this graph are directed**. For this problem Goal node using **Depth- Limited Search** (DLS) Where and the depth **limit is 1.**



⇨ Solutions:

Vertex Visited	Vertex in Stack	Graph
a	b c d	
a | b	c d	

a b c	d	
a b c d		
Cutoff: Goal **may exist**, but not found **within** the current depth limit.		

ITERATIVE DEEPENING SEARCH (IDS)

Iterative Deepening Search (IDS)

⇨ Iterative Deepening Search is a **search strategy** that combines the benefits of:

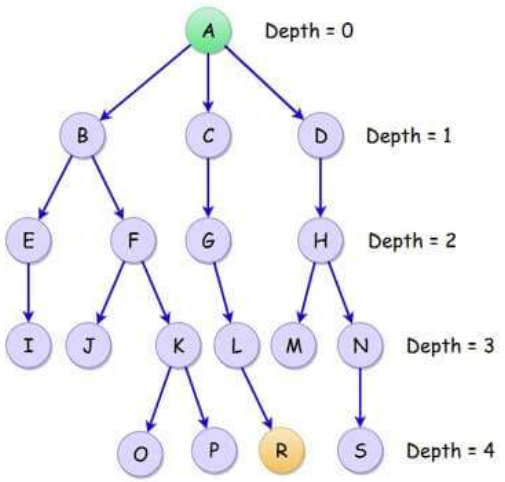
- **Depth-First Search (DFS)** (low memory usage)
- **Breadth-First Search (BFS)** (completeness and optimality for uniform step cost)

Note: It repeatedly performs depth-limited searches with increasing depth limits.

⇨ Core Idea

- Start with depth limit $\ell = 0$.
- Run **DLS** up to depth ℓ.
- If goal not found, increment ℓ and repeat.
- Stop when goal is found.

Example:

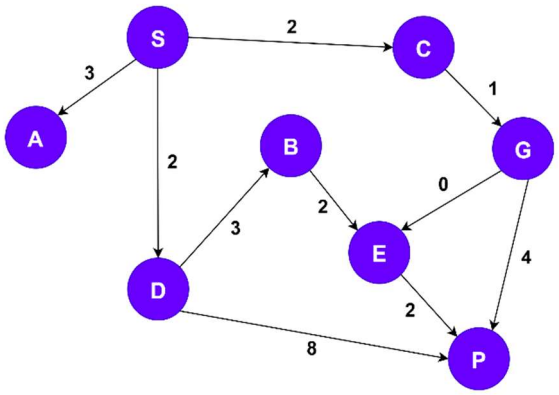
Graph	**IDS** (left to right)
	*DEPTH LIMITS* — IDDFS 0 — A 1 — A B C D 2 — A B E F C G D H 3 — A B E I F J K C G L D H M N 4 — A B E I F J K O P C G L R D H M N S

Properties of IDS

Property	Description
Completeness	Yes (will find solution if exists)
Optimality	Yes (if step cost = 1)
Time	$O(b^d)$, same as BFS
Space	O(d), same as DFS (much better than BFS)

Problem1:

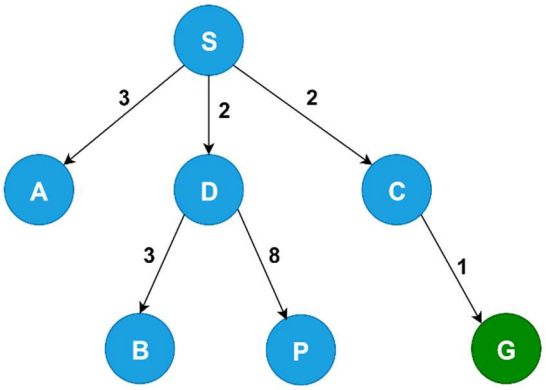
≫Show the simulation [frontier, explored set, path(solution), Path Cost, Tree] using Iterative depending. **Start node S** and **goal node G.** Insert the node in the frontier in alphabetic order.



Iterations	Depth	Frontier List	Explored List	Goal
1	0	[S]	[S]	Not Found
2	1	[A, C, D]	[S, A, C, D]	Not Found
3	2	[B, G, P]	[S, A, C, G, D, B, P]	**G found**

Term	Solution	Meaning
Explored	S, A, C, G, D, B, P	All nodes visited during the search
Solution Path	**S → C → G**	Actual path from start to goal
Path Cost	2 (S→C) + 1 (C→G) = **3**	Total weight of the solution path

Search Tree:



BIDIRECTIONAL SEARCH

Bidirectional Search

⇨ **Bidirectional Search** is a search algorithm that **runs two simultaneous searches**:

- One **forward** from the **initial state**
- One **backward** from the **goal state**

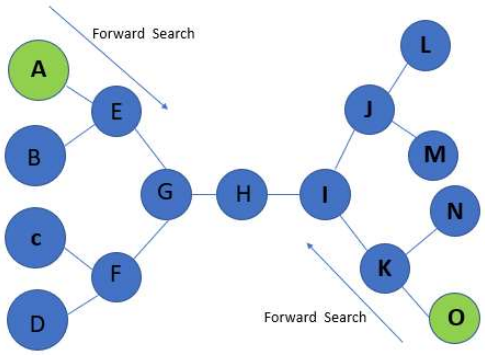
Note: The search stops when the two **searches meet**.

Algorithm Steps

1. Start two queues:
   - One from the **start node**
   - One from the **goal node**
2. Alternate between expanding:
   - Forward frontier from start
   - Backward frontier from goal
3. Stop when:
   - A node in the forward frontier also appears in the backward frontier.
4. Combine the two paths to form the complete solution.

Example:

Graph



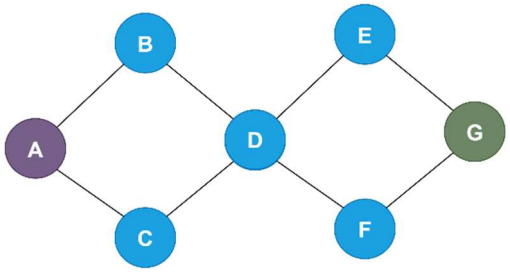
Bidirectional Search Steps

Forward Frontier		Backward Frontier	
Vertex Visited	Vertex in Queue	Vertex Visited	Vertex in Queue
A	E	O	K
A E	B G	O K	N I
A E B	G	O K N	I
A E B G	F H	O K N I	J H
A E B G F	H C D	O K N I J	H L M
A E B G F H	C D I	O K N I J H	L M G

Solution Path: A → E → B → G → F → H → J → I → N → K → O

Problem1:

≫Show the simulation using Bidirectional Search. **Start node A** and **goal node G.**



Bidirectional Search Steps			
Forward Frontier		**Backward Frontier**	
Vertex Visited	Vertex in Queue	Vertex Visited	Vertex in Queue
A	B C	G	E F
A B	C D	G E	F D
A B C	D	G E F	D
A B C D	E F	G E F D	B C
Solution Path: A → B → C → D → F → E → G			

Properties

Metric	Bidirectional Search	Breadth-First Search (BFS)
Time	$O\left(b^{\frac{d}{2}}\right)$	$O(b^d)$
Space	$O\left(b^{\frac{d}{2}}\right)$	$O(b^d)$
Completeness	Yes (if goal is reachable)	
Optimality	Yes (if BFS is used in both directions and edge cost = 1)	

BEST-FIRST SEARCH

Best-First Search (Greedy Search)

⇨ **Best-First Search** is an informed search algorithm that uses a **heuristic function** `h(n)` to select the **most promising node** to expand next.

Note: It expands the node that appears to be **closest to the goal**, based on the **heuristic estimate**.

➢ Heuristic Function (h(n))

- `h(n)` = Estimated cost from node `n` to the goal.
- It **does not** consider the cost from the start node.

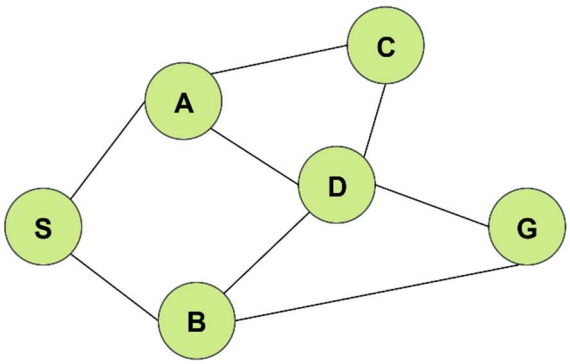
This is why it's called **Greedy Best-First Search** — it **greedily** chooses what looks best **now**, not considering the full path cost.

Algorithm Steps

1. Initialize a **priority queue** with the **start node**.
2. While the queue is not empty:
   - Remove the node with the **lowest** `h(n)` value.
   - If it's the **goal**, return the path.
   - Otherwise, expand the node and **insert its children** into the queue based on their `h(n)`.

Problem1:

≫ Consider **S** as the starting state and **G** is the goal state in given Graph. The heuristic values **h(n)** are provided beside the graph. Show the simulations for **Best First Search**.



H	V	h(H,V)
A	G	2
B	G	3
C	G	1
D	G	4
S	G	10
G	G	0

⇨ **Solutions:**

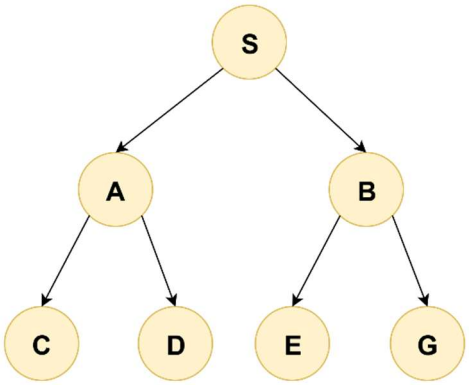
Iterations	Open	Close	Visit
1	Open [S]	Close []	NULL
2	Open [A, B]	Close [S]	S
3	Open [B, C, D]	Close [S, A]	A
4	Open [B, D]	Close [S, A, C]	C
5	Open [D, G]	Close [S, A, C, B]	B
6	Open [D]	Close [S, A, C, B, G]	G (GOAL)
Solution Path: S→ A → C → B→ G			

Properties of Best-First Search

Property	Value
Complete	Not always (may get stuck in loops)
Optimal	No (chooses based on h(n) only)
Time	$O(b^m)$ — depends on heuristic quality
Space	$O(b^m)$ — stores all nodes in memory

Problem2:

≫ Consider **S** as the starting state and **G** is the goal state in given Graph. The heuristic values **h(n)** are provided beside the graph. Show the simulations for **Best First Search**



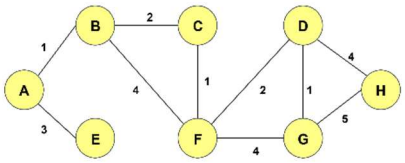
n	h(n)
A	2
B	3
C	1
D	4
S	10
E	6
G	0

Solutions:

Iterations	Open	Close	Visit
1	Open [S]	Close []	NULL
2	Open [A, B]	Close [S]	S
3	Open [B, C, D]	Close [S, A]	A
4	Open [B, D]	Close [S, A, C]	C
5	Open [D, G, E]	Close [S, A, C, B]	B
6	Open [D, E]	Close [S, A, C, B, G]	G (GOAL)
Solution Path: S⟶ A ⟶ C ⟶ B⟶ G			

☄ Problem3:

≫ Consider **A** as the starting state and **G** is the goal state in given Graph. The heuristic values **h(n)** are provided beside the graph. Show the simulations for **Best First Search**



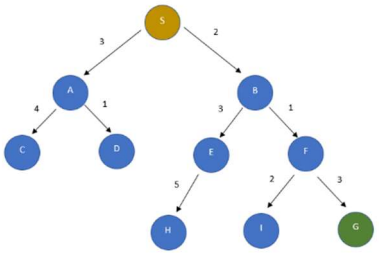
H	V	h(H,V)
A	G	9
B	G	7
C	G	4
D	G	1
E	G	10
F	G	3
H	G	5
G	G	0

⇨ Solutions:

Iterations	Open	Close	Visit
1	Open [A]	Close []	NULL
2	Open [B, E]	Close [A]	A
3	Open [E, C, F]	Close [A, B]	B
4	Open [E, C, D, G]	Close [A, B, F]	F
5	Open [E, C, D, H]	Close [A, B, F, G]	G (GOAL)
Solution Path: A → B → F → G			

☄ Problem4:

≫ Consider **S** as the starting state and **G** is the goal state in given Graph. The heuristic values **h(n)** are provided beside the graph. Show the simulations for **Best First Search**



node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

Iterations	Open	Close	Visit
1	Open [S]	Close []	NULL
2	Open [A, B]	Close [S]	S
3	Open [A, E, F]	Close [S, B]	B
4	Open [A, E, I, G]	Close [S, B, F]	F
5	Open [A, E, I]	Close [S, B, F, G]	G (GOAL)
Solution Path: S → B → F → G			

A* SEARCH ALGORITHM

A* Search Algorithm

⇨ **A* (A-Star) Search** is an informed search algorithm that finds the **least-cost path** to the goal using a combination of:

- `g(n)` = cost from start to current node `n`
- `h(n)` = heuristic estimate from `n` to goal

It uses:

$$f(n) = g(n) + h(n)$$

Note: A* selects the node with the **lowest estimated total cost** (`f(n)`) to reach the goal.

➢ Why A* is Powerful

- It balances:
  - **Cost so far** (g(n))
  - **Estimated cost to goal** (h(n))
- **Guarantees optimality**, if `h(n)` is **admissible** (never overestimates).

Algorithm Steps

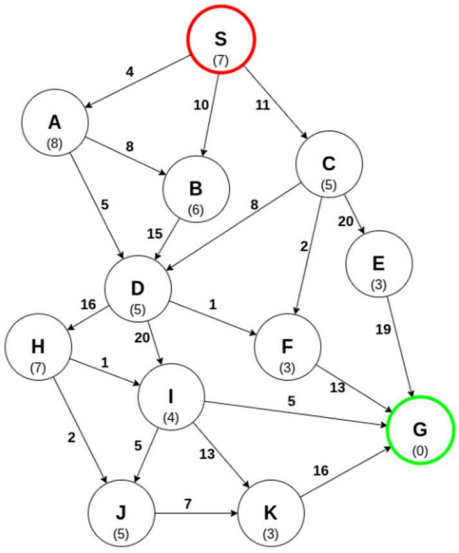
1. Add the **start node** to a **priority queue**, with `f(start) = g(start) + h(start).`
2. Loop:
   - Pick the node with the **lowest f(n)** from the queue.
   - If it is the goal, return the path.
   - Else, expand its children.
   - For each child:
     - Calculate `g(child)` and `f(child).`
     - Add to the queue if it has a lower cost path.

Properties of A*

Property	Description
Complete	Yes (if step cost ≥ ε > 0)
Optimal	Yes (if heuristic is **admissible** & **consistent**)
Time	$O(b^d)$ in worst case
Space	$O(b^d)$ — stores all paths in memory

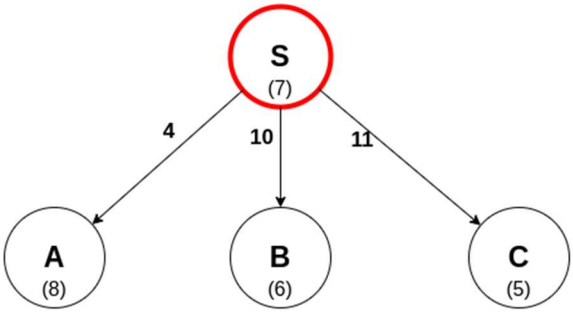
Problem1:

≫ Consider **S** as the starting state and **G** is the goal state in given Graph. The heuristic values **h(n)** is given in the **Node** and the step costs are given in the **edges**. Construct the search tree to find the **least cost path** from the start state S to the goal state G **using A* search** for the given graph. Clearly mention the **expand sequence** and, **path with path cost.**



⇨ Solutions:

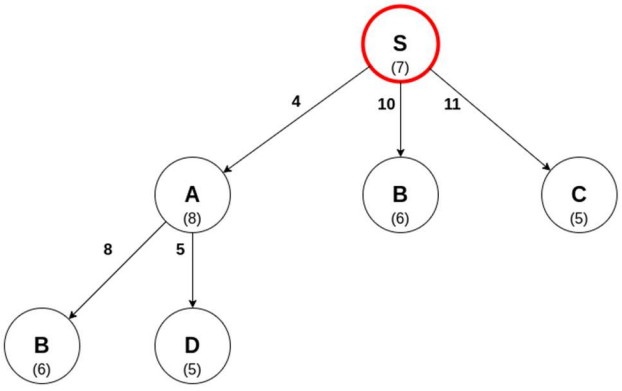
Exploring S:



Node[cost]
A[12]
B[16]
C[16]

Closed List
S

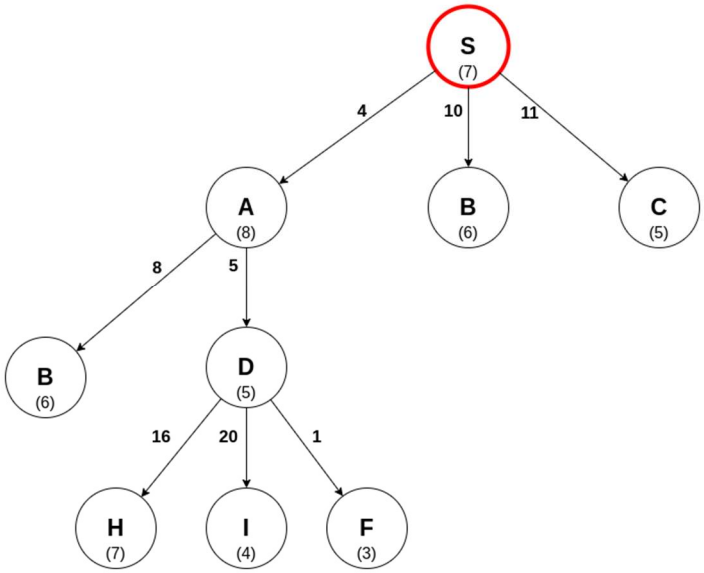
A is the current most promising path, so it is explored next:



Node[cost]
D[14]
C[16]
B[16]
B[18]

Closed List
S
A

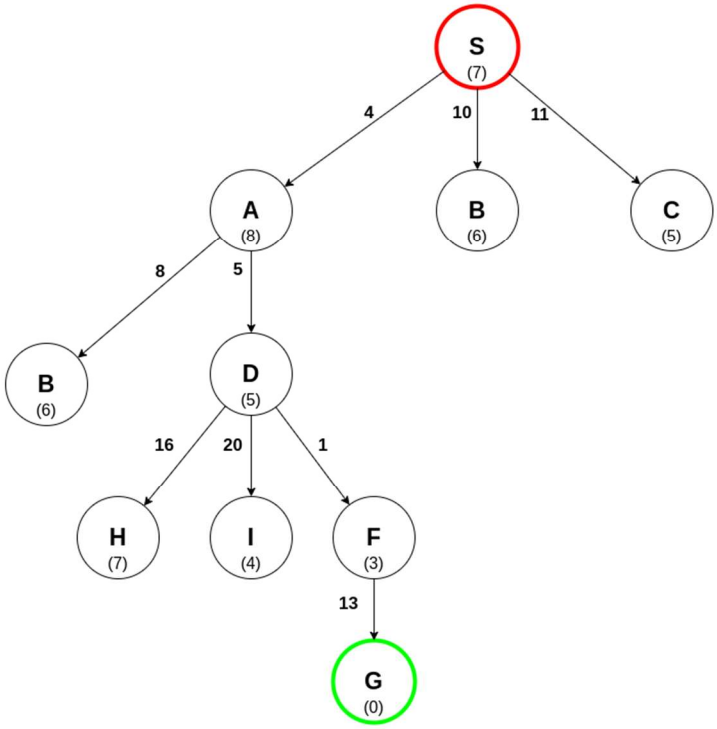
Exploring **D**:



Node[cost]
F[13]
C[16]
B[16]
H[32]
I[33]
B[18]

Closed List
S
A
D

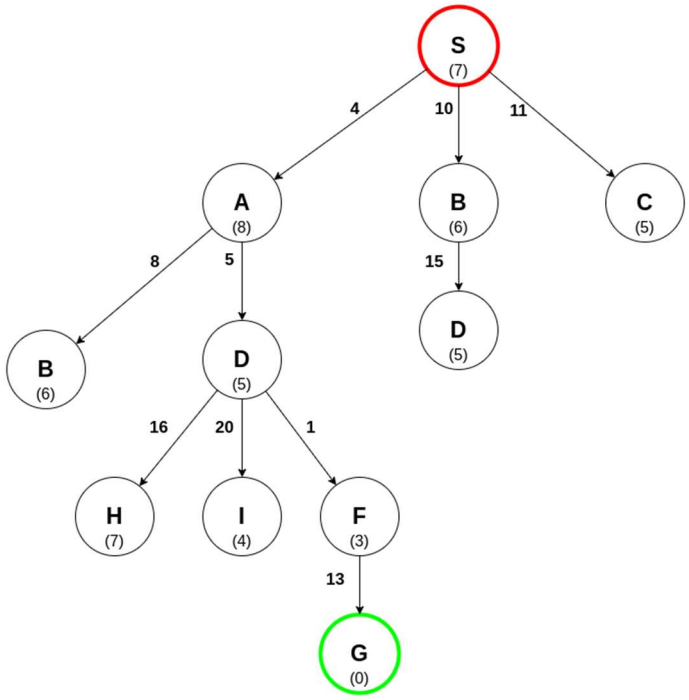
Exploring **F**:



Node[cost]
B[16]
C[16]
B[18]
H[32]
I[33]
G[23]

Closed List
S
A
D
F

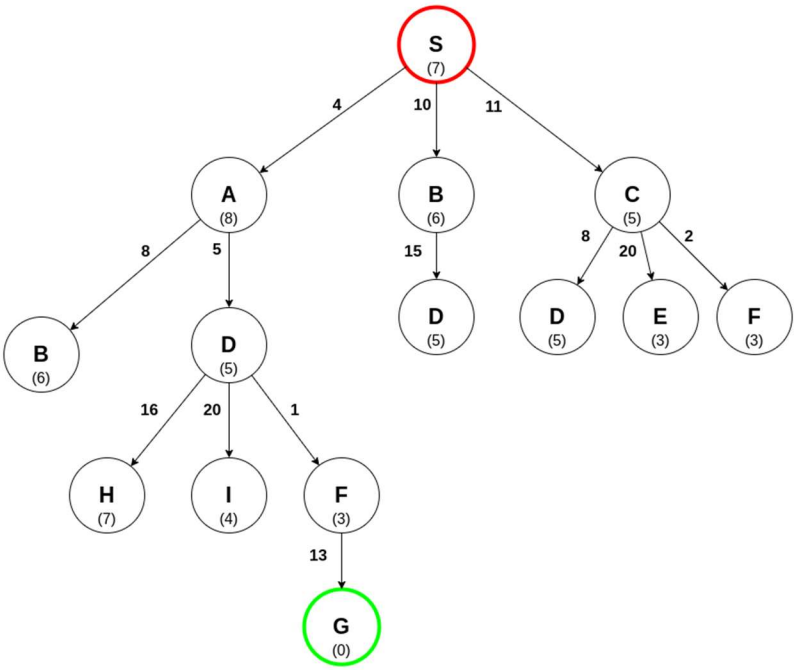
The goal node G is found but not yet explored, so the algorithm continues in case there's a shorter path. Node B appears twice in the open list (cost 16 from S, cost 18 from A); the lower-cost entry is explored next.



Node[cost]
C[16]
G[23]
B[18]
H[32]
I[33]

Closed List
S
A
D
F
B

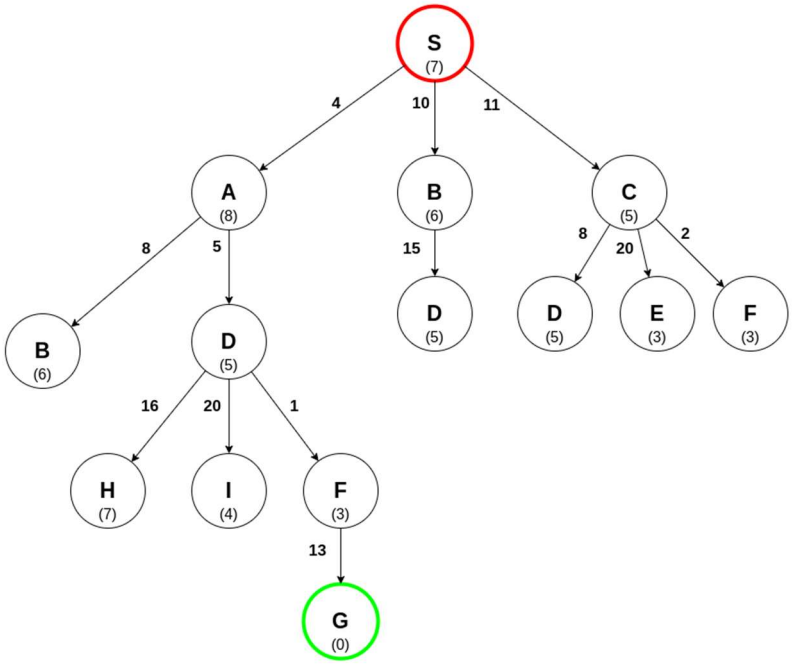
Exploring **C:**



Node[cost]
B[18]
G[23]
I[33]
H[32]
E[36]

Closed List
S
A
D
F
B
C

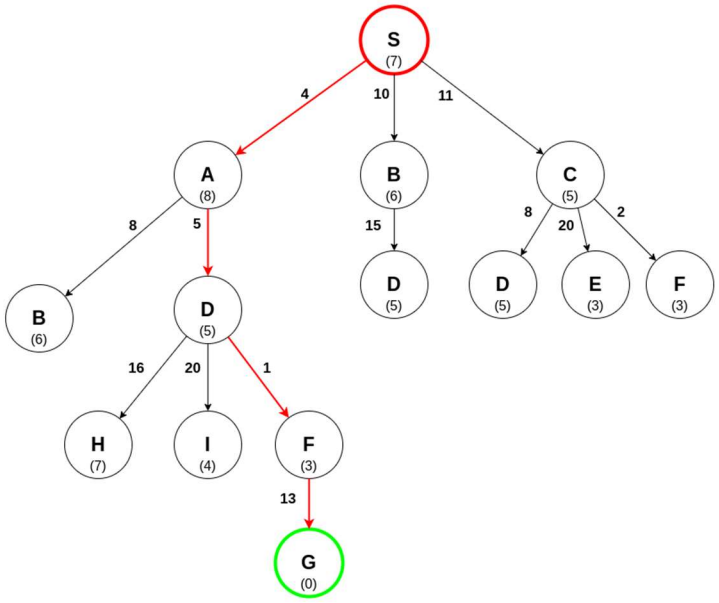
The next node in the open list is again **B**. However, because **B** has already been explored, meaning the shortest path to **B** has been found, it is not explored again, and the algorithm continues to the next candidate.



Node[cost]
G[23]
I[33]
H[32]
E[36]

Closed List
S
A
D
F
B
C

The next node to be explored is the **goal node G**, meaning the shortest path to G has been found! The path is constructed by tracing the graph backwards from G to S:



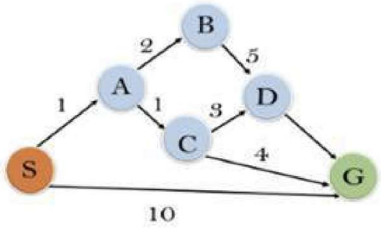
Node[cost]
I[33]
H[32]
E[36]

Closed List
S
A
D
F
B
C
G

Solution Path: S→ A → D → F → G	Solution Path Cost: 23

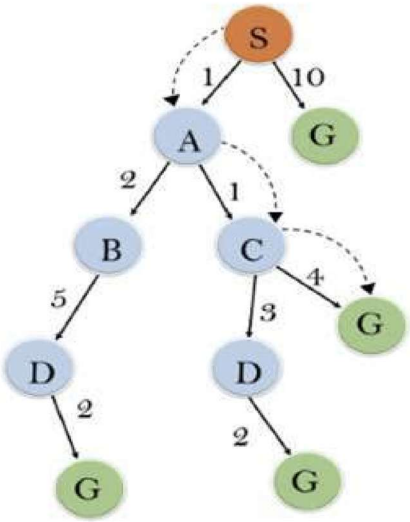
Problem2:

≫ Consider **S** as the starting state and **G** is the goal state in given Graph. The heuristic values **h(n)** are provided beside the graph and the step cost are labeled on the edges. Construct the search tree to find the **least cost path** from the start state S to the goal state G **using A* search** for the given graph. Clearly mention the **expand sequence** and, **path with path cost.**



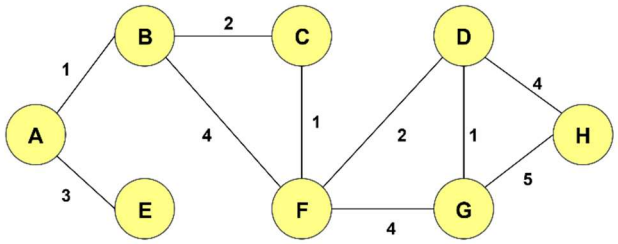
State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0

Iterations	Expand Node	Expand Sequence
1	S	$\{(S \rightarrow A, 4), (S \rightarrow G, 10)\}$
2	A	$\{(S \rightarrow A \rightarrow C, 4), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$
3	C	$\{(S \rightarrow A \rightarrow C \rightarrow G, 6), (S \rightarrow A \rightarrow C \rightarrow D, 11), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$
4	G	$(S \rightarrow A \rightarrow C \rightarrow G, 6)$
Solution Path: S→ A → C → G		**Solution Path Cost: 6**
Tree:		



Problem3:

≫ Consider **A** as the starting state and **G** is the goal state in given Graph. The heuristic values **h(n)** are provided beside the graph and the step cost are labeled on the edges. Show the simulations for **A* Search**.



H	V	h(H,V)
A	G	9
B	G	7
C	G	4
D	G	1
E	G	10
F	G	3
H	G	5

⇨ Solutions:

Iterations	Expand Node	Expand Sequence
1	A	$\{(A \rightarrow B, 8), (A \rightarrow E, 13)\}$
2	B	$\{(A \rightarrow B \rightarrow C, 7), (A \rightarrow B \rightarrow F, 8), (A \rightarrow E, 13)\}$
3	C	$\{(A \rightarrow B \rightarrow C \rightarrow F, 7), (A \rightarrow B \rightarrow F, 8), (A \rightarrow E, 13)\}$
4	F	$\{(A \rightarrow B \rightarrow C \rightarrow F \rightarrow D, 7), (A \rightarrow B \rightarrow C \rightarrow F \rightarrow G, 8),$ $(A \rightarrow B \rightarrow F, 8), (A \rightarrow E, 13)\}$
5	D	$\{(A \rightarrow B \rightarrow C \rightarrow F \rightarrow D \rightarrow G, 7), (A \rightarrow B \rightarrow C \rightarrow F \rightarrow D \rightarrow H, 15),$ $(A \rightarrow B \rightarrow C \rightarrow F \rightarrow G, 8), (A \rightarrow B \rightarrow F, 8), (A \rightarrow E, 13)\}$
6	G	$(A \rightarrow B \rightarrow C \rightarrow F \rightarrow D \rightarrow G, 7)$
Solution Path: A→ B→ C → F→ D→ G		**Solution Path Cost: 7**

8-PUZZLE PROBLEM

8-puzzle problem (Uninformed Search)

Problem1:

≫ Given an initial state of an 8-puzzle problem and the final state to be reached, find the solution path from the initial state to the final state **using an appropriate uninformed search algorithm (e.g., BFS, UCS, or IDS)** and show the sequence of moves required.

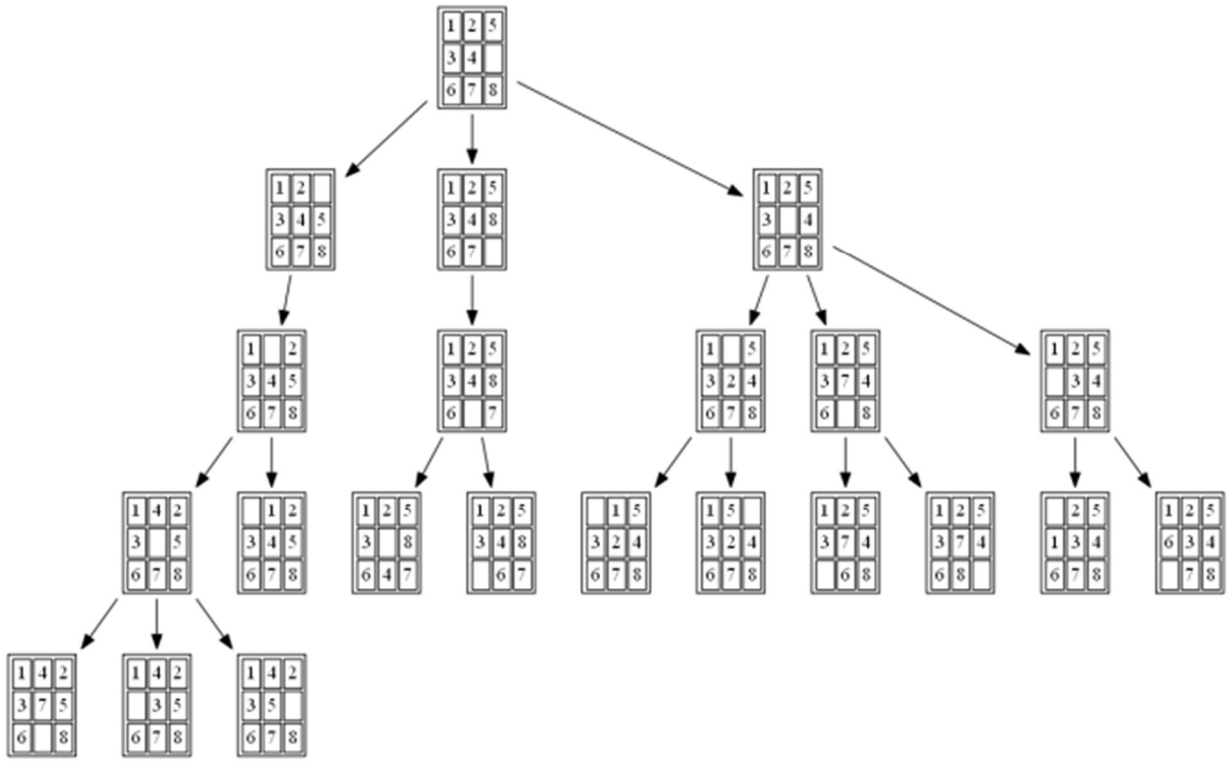
1	2	5
3	4	
6	7	8

Initial state

1	4	2
3	5	
6	7	8

Final state

Solution- using BFS:



8-puzzle problem (Informed Search)

Problem1:

≫ Given an initial state of a 8-puzzle problem and final state to be reached-

2	8	3
1	6	4
7		5

Initial State

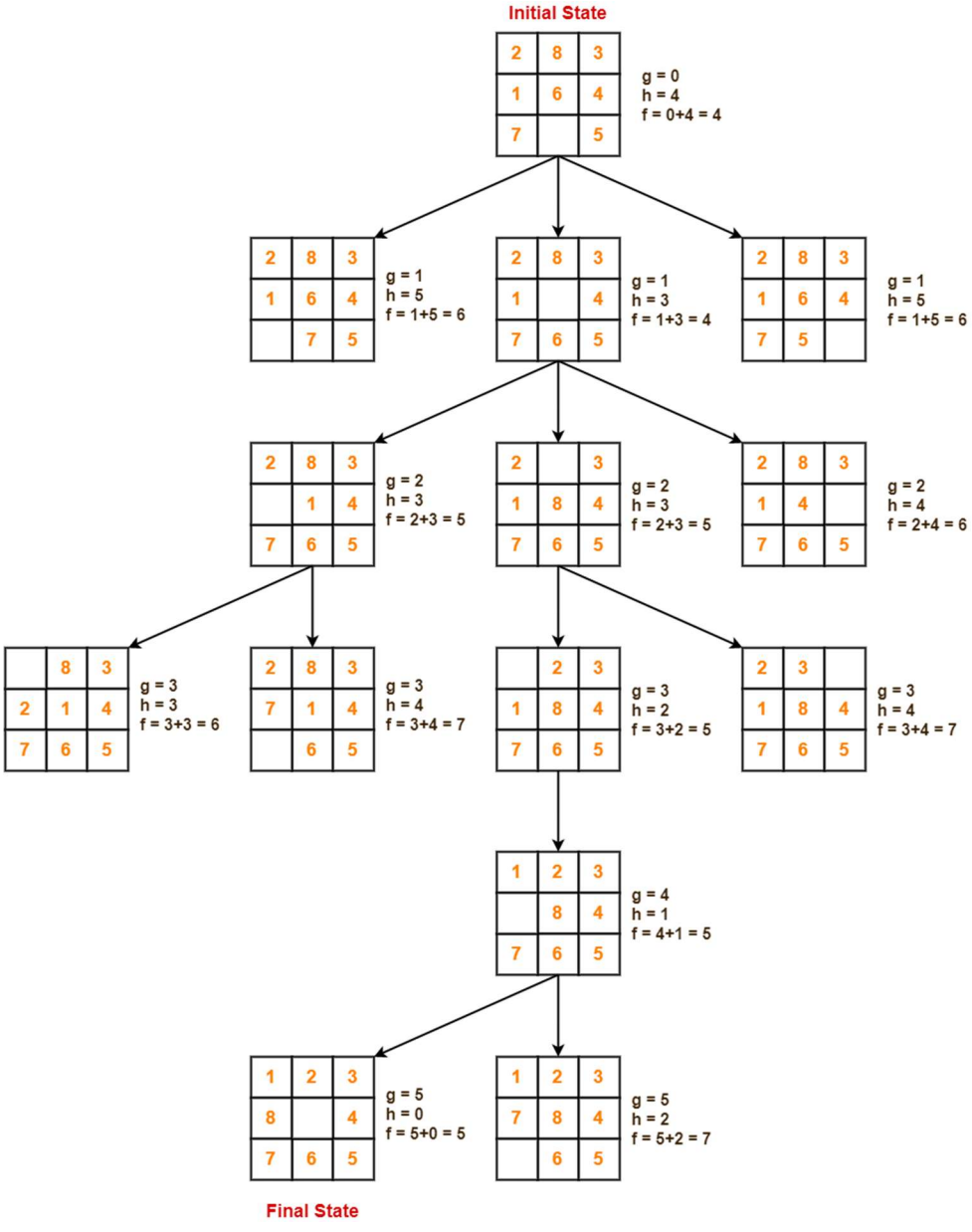
1	2	3
8		4
7	6	5

Final State

- Find the most cost-effective path to reach the final state from initial state using A* Algorithm.
- Consider $g(n) = \textbf{Depth of node}$ & $h(n) = \textbf{Number of misplaced tiles}$.

Solution-

- A* Algorithm maintains a tree of paths originating at the initial state.
- It continues until final state is reached.



Problem2:

≫ Find the heuristics for the 8-puzzel problem using the initial state of the given puzzle in Figure.

5	4	
6	1	8
7	3	2

Initial state

1	2	3
8		4
7	6	5

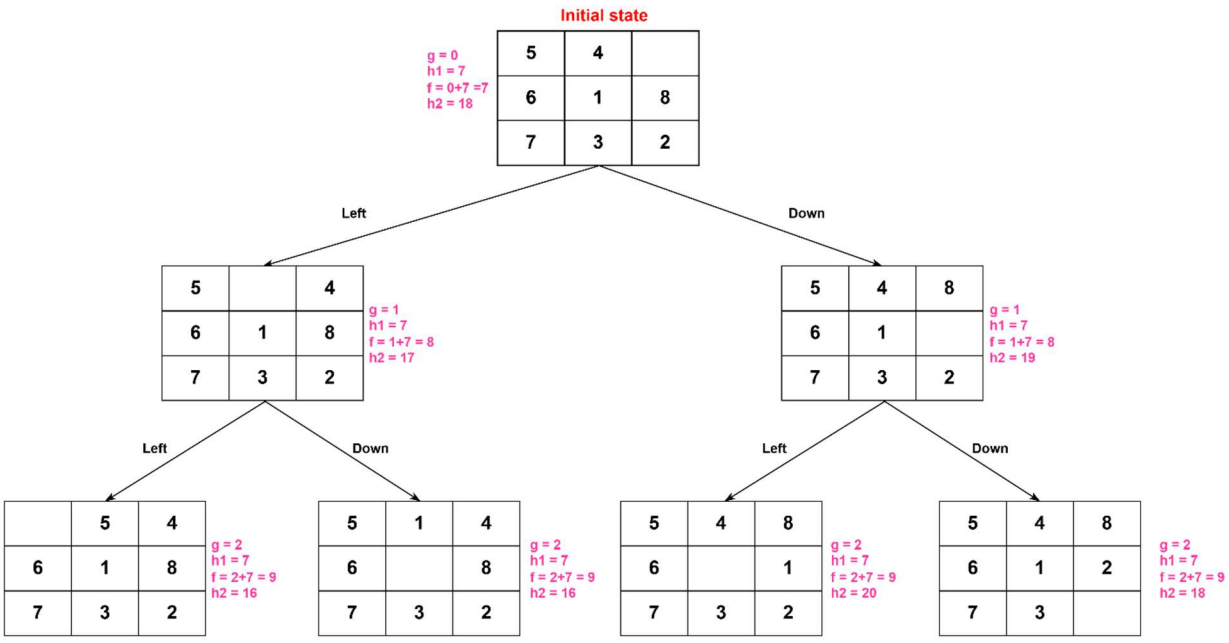
Final state

i. $\mathbf{h_1}$ = The number of misplaced tiles (Count how many tiles are not in their goal position)
ii. $\mathbf{h_2}$ = The sum of distances of the tiles from their goal position (use Manhattan distance for each tile)
iii. Draw the partial state space tree for the 8-puzzle problem based on the figure, up to depth level 2. Start from the initial state and generate all possible states up to two moves (depth = 2)

(i & ii)

h_1 = number of misplaced tiles	h_2 = sum of Manhattan distances
$h_1 = 1_5 + 1_4 + 1_8 + 1_2 + 1_3 + 1_6 + 1_1 = 7$	$h_2 = 4_5 + 2_4 + 2_8 + 3_2 + 3_3 + 2_6 + 2_1 = 18$

(iii)



Practice Problem

Problem1:

≫ Draw the schematic diagram of a Utility-Based Agent and describe its working procedure.

Solution-

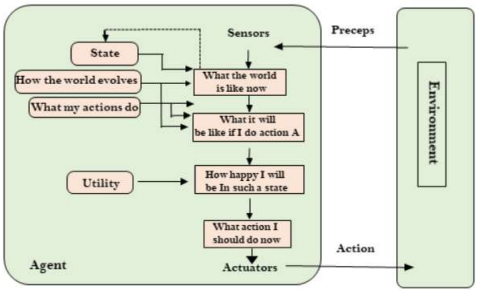


Figure 2.14 A model-based, utility-based agent. It uses a model of the world, along with a utility function that measures its preferences among states of the world. Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.

Problem2:

≫ Explain the properties of "Completeness" and "Cost Optimality" for the DFS and BFS search algorithms

Completeness:

⇨ **A search algorithm is** complete **if it is guaranteed to find a solution** if one exists**.**

Algorithm	Completeness	Reason
BFS	Yes	Explores all nodes level-by-level, so it will reach the goal if reachable in a finite graph.
DFS	No	May go infinitely deep in infinite state spaces or get stuck in loops without reaching the goal.

Cost Optimality:

⇨ A search algorithm is cost-optimal if it always returns the solution with the **lowest path cost**.

Algorithm	Cost Optimal	Reason
BFS	Yes (when all step costs are equal)	Finds the shallowest (fewest moves) solution, which is also the least cost if each step has cost = 1.
DFS	No	May find a deeper (more costly) solution before a shorter one, so not guaranteed optimal.

Problem3:

≫ Explain the "Completeness" and "Optimality" properties of **Iterative Deepening Search** algorithm.

Completeness:

- An algorithm is **complete** if it guarantees to find a solution **if one exists**.
- **IDS: Complete**
  - Because it performs **Depth-Limited Search** repeatedly with increasing limits ($\ell = 0, 1, 2, \ldots$), it will eventually reach the shallowest goal node.
  - Works for **finite state spaces** and step cost > 0.

Optimality:

- An algorithm is **optimal** if it always finds the **least-cost (or shallowest)** solution.
- **IDS: Optimal** (for **unit step costs**)
  - Because it finds the goal **at the smallest depth first**, just like BFS.
  - If costs vary (non-unit), IDS is **not guaranteed optimal** — in that case, UCS is used.

Problem4:

≫ Develop PEAS description of the task environment for a Candy Sorting Robot that can sort candies to the appropriate jars.

PEAS Element	Description
P – Performance Measure	Accuracy, Speed, Safety, Efficiency
E – Environment	Conveyor, Sorting Table, Jars, Lighting
A – Actuators	Robotic Arm, Conveyor Motor, Jar Mechanism
S – Sensors	Camera, Weight Sensor, Position Sensor, Proximity Sensor

Problem5:

≫ Describe the Turing Test that can be used to test the **'Acting humanly'** perspective.

Turing Test – Acting Humanly:

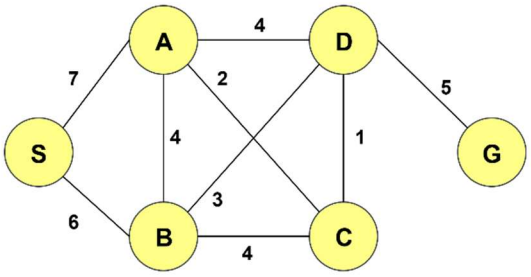
⇨ Proposed by **Alan Turing (1950)** to check if a machine can **act like a human**.
In the test:

- A **human judge** chats (text only) with a **human** and a **machine**.
- The judge asks any questions and tries to guess which is the machine.
- If the judge **can't reliably tell** which is which, the machine **passes**.

It matches the **"Acting Humanly"** perspective because it focuses on **human-like responses** rather than how the machine thinks.
Passing doesn't prove the machine is intelligent — only that it can **mimic human behavior** well enough to fool a person.

Problem6:

≫Simulate Depth-First Search (DFS) on the graph shown in below, **starting from the node S** and **aiming to reach the goal node G**. (push nodes into the frontier in alphabetic order)

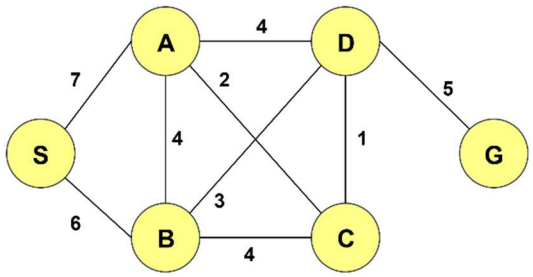


Steps	Vertex in Stack	Pop & Expand	Vertex Visited	Goal Node
1	S	S	Empty	Null
2	B A	B	S	**S** Not Goal
3	D C A	D	S B	**B** Not Goal

4	G C A	G	S B D	**D** Not Goal
5	C A	Null	S B D G	**G** Goal

Problem7:

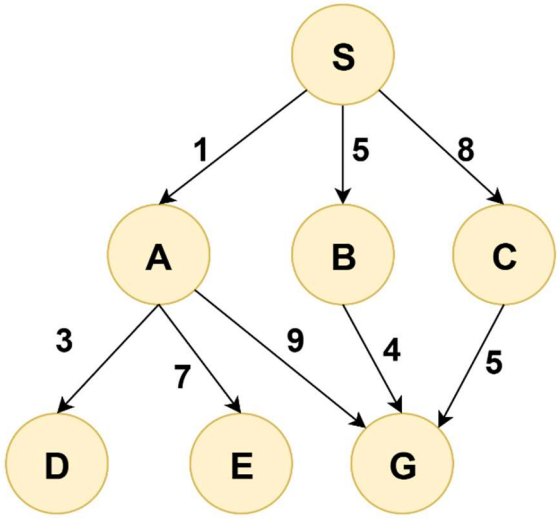
≫Simulate Breadth-First Search (BFS) on the graph shown in below, **starting from the node S** and **aiming to reach the goal node G**. (push nodes into the frontier in alphabetic order)



Steps	Vertex in Queue	Dequeue & Expand	Vertex Visited	Goal Node
1	S	S	Empty	Null
2	A B	A	S	**S** Not Goal
3	B C D	B	S A	**A** Not Goal
4	C D	C	S A B	**B** Not Goal
5	D	D	S A B C	**C** Not Goal
6	G	G	S A B C D	**D** Not Goal
7	Empty	Null	S A B C D G	**G** Goal

Problem8:

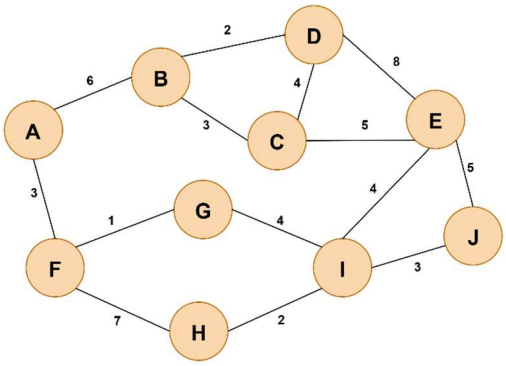
≫ Consider the given graph below, where you need to reach the destination node G starting from node S. Node {A, B, C, D & E} are the intermediate nodes. Your task is to find the path from S to the destination node G with the least cumulative cost using Uniform Cost Search. Show the steps of searching the best path.



Step	Frontier List	Expand List	Explored List
1	{(S,0)}	S	NULL
2	{(S-**A**,1), (S-**B**,5), (S-**C**,8)}	A	{S}
3	{(S-**B**,5), (S-**C**,8), (S-A-**D**,4), (S-A-**E**,8), (S-A-**G**,10)}	B	{S, A}
4	{(S-**C**,8), (S-A-**D**,4), (S-A-**E**,8), (S-A-**G**,10), (S-B-G,9)}	D	{S, A, B}
5	{(S-**C**,8), (S-A-**E**,8), (S-A-**G**,10), (S-B-**G**,9)}	C	{S, A, B, D}
6	{(S-A-**E**,8), (S-A-**G**,10), (S-B-**G**,9), (S-C-**G**,13)}	E	{S, A, B, D, C}
7	{(S-A-**G**,10), (S-B-**G**,9), (S-C-**G**,13)}	G	{S, A, B, D, C, E}
8	(S-B-**G**,9)	NULL	{S, A, B, D, C, E, G} # GOAL Found!
Solution Path: S→ B→ G		**Solution Path Cost: 9**	
Travers Path: S→ A→ B→ D→ C→ E→ G			

Problem9:

≫Perform Uniform Cost Search starting (UCS) form **Node A** with the goal of reaching Node **J.** If priority is same for different nodes, tie break using alphabetical order.



Step	Frontier List	Expand List	Explored List
1	{(A,0)}	A	NULL
2	{(A-**B**,**6**), (A-**F**,**3**)}	F	{A}
3	{(A-**B**,**6**), (A-F-**G**,**4**), (A-F-**H**,**10**)}	G	{A, F}
4	{(A-**B**,**6**), (A-F-**H**,**10**), (A-F-G-**I**,**8**)}	B	{A, F, G}
5	{(A-F-**H**,**10**), (A-F-G-**I**,**8**), (A-B-**D**,**8**), (A-B-**C**,**9**)}	D	{A, F, G, B}
6	{(A-F-**H**,**10**), (A-F-G-**I**,**8**), (A-B-**C**,**9**), (A-B-D-**E**,**16**)}	I	{A, F, G, B, D}
7	{(A-F-**H**,**10**), (A-B-**C**,**9**), (A-B-D-**E**,**16**), (A-F-G-I-**E**,**12**), (A-F-G-I-**J**,**11**)}	C	{A, F, G, B, D, I}
8	{(A-F-**H**,**10**), (A-B-D-**E**,**16**), (A-F-G-I-**E**,**12**), (A-F-G-I-**J**,**11**), (A-B-C-**E**,**14**)}	H	{A, F, G, B, D, I, C}
9	{(A-B-D-**E**,**16**), (A-F-G-I-**E**,**12**), (A-F-G-I-**J**,**11**), (A-B-C-**E**,**14**), ~~(A-F-H-**I**,**12**)~~}	J	{A, F, G, B, D, I, C, H}
10	(A-F-G-I-**J**,**11**)	NULL	{A, F, G, B, D, I, C, H, J} **# GOAL Found!**
Solution Path: A→ F→ G → I→ J		**Solution Path Cost: 11**	
Travers Path: A→ F→ G→ B→ D→ I→ C→ H→ J			

Problem10:

≫Which algorithm is comparatively faster (Expend less node) A* or UCS? Briefly explain.

⇨ **A*** is generally faster than **UCS** because it uses both the path cost from the start and a heuristic estimate to the goal, guiding the search more efficiently. This focused search means **A*** expands fewer nodes.

On the other hand, **UCS** only considers the path cost from the start, ignoring the goal direction, so it may explore many unnecessary nodes.

With an admissible and consistent heuristic, **A*** finds the optimal path faster by reducing the search space compared to **UCS**.

Problem11:

≫ The A* algorithm is complete and optimal, why?

⇨ **Complete:** It always finds a solution if one exists, as it systematically explores nodes by increasing estimated total cost and never ignores any possible paths.

⇨ **Optimal:** When using an **admissible heuristic** (one that never overestimates the true cost to the goal), A* guarantees the found path is the least-cost (optimal) path because it expands nodes in order of the lowest estimated total cost (actual cost so far + heuristic).

Problem12:

≫ The heuristic based algorithm in which the objective function is f(n) = (2- w)g(n) + wh(n) What kind of search does this perform when w = 0? When w = 1? When w =2?

When $w = 0$:

$$f(n) = (2 - 0)g(n) + 0 \times h(n) = 2g(n)$$

Since h(n) is ignored, the algorithm only considers the path cost so far. This behaves like **Uniform Cost Search (UCS)** (the factor 2 is just a constant scaling and does not affect the order of node expansions).

When $w = 1$:

$$f(n) = (2 - 1)g(n) + 1 \times h(n) = g(n) + h(n)$$

This is exactly the classic **A*** search formula.

When $w = 2$:

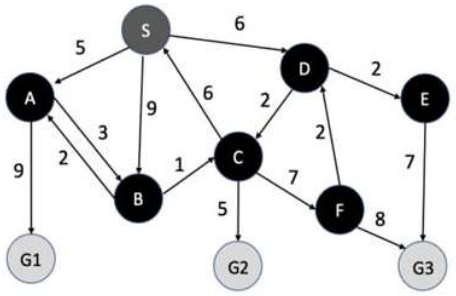
$$f(n) = (2 - 2)g(n) + 2 \times h(n) = 2h(n)$$

The algorithm ignores the path cost so far and only uses the heuristic. This behaves like a **Greedy Best-First Search**

Problem11:

(a) Consider the given graph where you need to reach any of the destination nodes {G1, G2, G3} starting from node S. Find the path from S to any of the destination nodes with the least cumulative cost using a Uniform Cost Search. (Lec 3.1)

(b) Is it possible to find the solution for the given graph using A* search? Explain your answer. [Lec 3.2]



(a)

Step	Frontier List	Expand List	Explored List
1	{(S,0)}	S	NULL
2	{(S-A,5), (S-B,9), (S-D,6)}	A	{S}
3	{(S-B,9), (S-D,6), (S-A-G1,14), (S-A-B,8)}	D	{S, A}
4	{(S-B,9), (S-A-G1,14), (S-A-B,8), (S-D-C,8), (S-D-E,8)}	B	{S, A, D}
5	{~~(S-B,9),~~ (S-A-G1,14), (S-D-C,8), (S-D-E,8), (S-A-B-C,9)}	C	{S, A, D, B}
6	{(S-A-G1,14), (S-D-E,8), ~~(S-A-B-C,9),~~ (S-D-C-G2,13), (S-D-C-F,15)}	E	{S, A, D, B, C}
7	{(S-A-G1,14), (S-D-C-G2,13), (S-D-C-F,15) (S-D-E-G3,15)}	G2	{S, A, D, B, C, E}
8	(S-D-C-G2,13)	NULL	{S, A, D, B, C, E, G2} **# GOAL Found!**
Solution Path: S→ D→ C → G2		**Solution Path Cost: 13**	
Travers Path: S→ A→ D→ B→ C→ E→ G2			

(b)

Yes, if we use an **admissible heuristic** (never overestimates the cost to the nearest goal). For multiple goals, use $h(n)$ =min of estimates to each goal. With $h = 0$, A* becomes UCS and still finds the optimal path.

Problem12:

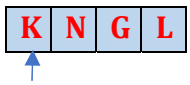
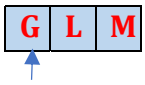
≫ Consider a grid where S is the starting state and G is the goal state. Movement is allowed in four directions: top, left, down, tight. provided the adjacent cell is an open box. Black boxes are obstacles and cannot be entered. You are to **construct the search tree** and solve the problem using: BFS. DFS. UCS

The default cost to move from or o another is **1**. If the target cell is on the boundary of the of the grid (but not a corner) the actions cost becomes **2.** If the target cell is in the corner, the actions cost become **3.**

S	A	B	
C		D	E
F	H	I	
	J	K	L
M	N	G	O

Using BFS:

Steps	Vertex in Queue	Expand	Explored List	Path
1	S	S	Empty	Null
2	A C	A	S	S
3	C B	C	S A	**S-A**
4	B F	B	S A C	**S-C**
5	F D	F	S A C B	**S-A-B**
6	D H	D	S A C B F	**S-C-F**
7	H E I	H	S A C B F D	**S-A-B-D**
8	E I J	E	S A C B F D H	**S-C-F-H**

9		I	S A C B F D H E	**S-A-B-D**
10		J	S A C B F D H E I	**S-C-F-H-I**
11		K	S A C B F D H E I J	**S-C-F-H-J**
12		N	S A C B F D H E I J K	**S-C-F-H-I-K**
13		G	S A C B F D H E I J K N	**S-C-F-H-J-N**
14		L	S A C B F D H E I J K N G	**S-C-F-H-I-K-G** **goal**
$\boldsymbol{Cost}$: $\mathbf{0+2+2+1+1+1+2=9}$				

Using DFS:

Steps	Vertex in Stack	Expand	Explored List	Path
1	**S**	S	**Empty**	Null
2	**C** **A**	C	S	S
3	**F** **A**	F	S C	S-C
4	**H** **A**	H	S C F	S-C-F
5	**J** **I** **A**	J	S C F H	S-C-F-H

6	N K I A	N	S C F H J	S-C-F-H-J
7	G M K I A	G	S C F H J N	S-C-F-H-J-N
8	O M K I A	O	S C F H J N G	S-C-F-H-J-N-G goal
$Cost: 0+2+2+1+1+2+2 = 10$				

Using UCS:

Steps	Vertex in Queue	Expand	Explored List	Path
1	(S,0)	S	Empty	Null
2	(C,2), (A,2)	A	S	S=0
3	(C,2), (B,4)	C	S A	**S-A=2**
4	(F,4), (B,4)	B	S A C	**S-C=2**
5	(F,4), (D,5)	F	S A C B	**S-A-B=4**
6	(D,5), (H,5)	D	S A C B F	**S-C-F=4**

7	(H,5), (I,6), (E,7)	H	S A C B F D	**S-A-B-D=5**
8	(J,6), (I,6), (E,7)	I	S A C B F D H	**S-C-F-H=5**
9	(J,6), (K,7), (E,7)	J	S A C B F D H I	**S-A-B-D-I=6**
10	(K,7), (E,7)	E	S A C B F D H I J	**S-C-F-H-J=6**
11	(K,7)	K	S A C B F D H I J E	**---------**
12	(G,9), (L,9)	G	S A C B F D H I J E K	**S-A-B-D-I-K=7**
13	-	-	S A C B F D H I J E K G	**S-A-B-D-I-K-G=9 goal**
$\boldsymbol{Cost}: \mathbf{0+2+2+1+1+1+2=9}$				

Problem13:

≫ What is the difference between tree search and graph search?

Tree Search – Expands nodes without tracking visited states. This can cause repeated state exploration and infinite loops in cyclic graphs. Uses less memory but may waste time revisiting nodes. Not always complete or optimal in graphs.

Graph Search – Maintains an *explored set* (visited states list) to avoid revisiting nodes. Prevents loops, improves efficiency in cyclic or large graphs, but requires more memory. Can be complete and optimal depending on the search strategy (e.g., BFS, UCS).

Problem14:

≫ What is the difference between goal agent and the utility agent

Aspect	Goal-Based	Utility-Based
Decision	Achieves a goal.	Maximizes utility value.
Evaluation	Success/failure.	Quality of outcome.
Flexibility	Any goal state is fine.	Chooses best among options.
Example	Deliver package anywhere on time.	Deliver package fastest & safest.

Problem15:

Design a painting agent for a 12×12 grid (144 squares). The agent may **move its arm to another square only after fully painting its current square**.

(a) Formulate this as a search problem by specifying the **initial state, actions, transition model, goal test, and path cost**.

(b) **Calculate and justify the total state-space size**.

(c) Classify the task environment as: **single vs. multi-agent, fully vs. partially observable, deterministic vs. stochastic, dynamic vs. static, episodic vs. sequential** (justify briefly).

(a)

Problem Formulation:

- **State:** Agent's position **(x, y)** and the set of painted squares **P**.
- **Initial State:** Start at **(1, 1)** with no painted squares.
- **Actions:**
  - If current square is unpainted → **Paint**.
  - If painted → **Move** (North, South, East, West) within grid.
- **Transition Model:**
  - **Paint** → mark square painted, stay in place.
  - **Move** → go to neighbor square, painted set unchanged (only from a painted square).
- **Goal Test:** All **144 squares** painted.
- **Path Cost:** 1 per action (paint or move).

(b)

State-space size

- **Positions:** 12×12 = 144 possible locations.
- **Paint patterns:** Each square painted/unpainted ⇒ 2^{144} possibilities.
- **Total states:** 144×2^{144}
- **Note:** "Paint-before-move" limits actions but not state count; many states may be unreachable.

(c)

Environment properties

- **Single vs. Multi-agent: Single-agent** (only the painter acts).
- **Observability: Fully observable** (assumed the agent can sense its position and which squares are painted).
- **Determinism: Deterministic** (Paint and Move have predictable outcomes).
- **Dynamics: Static** (environment doesn't change unless the agent acts).
- **Episodic vs. Sequential: Sequential** (current choices affect future options via the painted set and position).
- **Discrete:** Yes (discrete states, actions, and time steps).
- **Known vs. Unknown model (optional): Known**, if grid layout and rules are given.